

Multimedia

Vulkan API

Brno University of Technology, Faculty of Information Technology
Božetěchova 1/2, 612 66 Brno - Královo Pole

Matej Karas
ikaras@fit.vutbr.cz

Today



- OpenGL vs Vulkan
- Brief history of Vulkan
- Fly through Vulkan `hello triangle`
- Useful Vulkan extensions/libraries
- ~~Vulkan Video~~

Why Vulkan?



1. Performance
2. Performance
3. Performance

Why Vulkan?



- OpenGL way too old/bloated
- Current HW
- Performance/low overhead (very explicit)
- Cross-platform
- Multithreaded
- Proton/DXVK
 - Steam Deck

OpenGL 1.x

```
#include <iostream>
#include <GL/gl.h>
#include <GL/glu.h>

#define RESOLUTION_WIDTH 400
#define RESOLUTION_HEIGHT 400

void initGL(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    gluPerspective(45.0f, (GLfloat)RESOLUTION_WIDTH / RESOLUTION_HEIGHT, 0.1f, 100.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void render(void)
{
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_TRIANGLES);
        glVertex3f(0.0f, 0.0f, 0.0f);
        glVertex3f(1.0f, 0.0f, 0.0f);
        glVertex3f(0.5f, 0.8660254f, 0.0f);
    glEnd();

    glutSwapBuffers();
}

int main(int argc, char** argv)
{
    SDL_Init(SDL_INIT_VIDEO);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_OPENGL);
    SDL_GL_SetAttribute(SDL_GL_DEPTH_SIZE, 24);
    SDL_GL_SetAttribute(SDL_GL_STENCIL_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);
    SDL_GL_SetAttribute(SDL_GL_SWAP_CONTROL, 1);

    SDL_Window* window = SDL_CreateWindow(argv[1], 0, 0, RESOLUTION_WIDTH, RESOLUTION_HEIGHT, SDL_WINDOW_OPENGL);
    SDL_GLContext glContext = SDL_GL_CreateContext(window);

    SDL_SetRenderTarget(window, glContext);

    initGL();

    while(1)
    {
        SDL_Event e;
        while(SDL_PollEvent(&e))
        {
            if(e.type == SDL_QUIT)
            {
                return 0;
            }
        }

        render();
        SDL_GL_SwapBuffers();
    }
}
```

```
#include <iostream>
#include <GL/gl.h>
#include <GL/glu.h>

#define RESOLUTION_WIDTH 400
#define RESOLUTION_HEIGHT 400

void initGL(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    gluPerspective(45.0f, (GLfloat)RESOLUTION_WIDTH / RESOLUTION_HEIGHT, 0.1f, 100.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void render(void)
{
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_TRIANGLES);
        glVertex3f(0.0f, 0.0f, 0.0f);
        glVertex3f(1.0f, 0.0f, 0.0f);
        glVertex3f(0.5f, 0.8660254f, 0.0f);
    glEnd();

    glutSwapBuffers();
}

int main(int argc, char** argv)
{
    SDL_Init(SDL_INIT_VIDEO);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_OPENGL);
    SDL_GL_SetAttribute(SDL_GL_DEPTH_SIZE, 24);
    SDL_GL_SetAttribute(SDL_GL_STENCIL_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);
    SDL_GL_SetAttribute(SDL_GL_SWAP_CONTROL, 1);

    SDL_Window* window = SDL_CreateWindow(argv[1], 0, 0, RESOLUTION_WIDTH, RESOLUTION_HEIGHT, SDL_WINDOW_OPENGL);
    SDL_GLContext glContext = SDL_GL_CreateContext(window);

    SDL_SetRenderTarget(window, glContext);

    initGL();

    while(1)
    {
        SDL_Event e;
        while(SDL_PollEvent(&e))
        {
            if(e.type == SDL_QUIT)
            {
                return 0;
            }
        }

        render();
        SDL_GL_SwapBuffers();
    }
}
```

```
#include <iostream>
#include <GL/gl.h>
#include <GL/glu.h>

#define RESOLUTION_WIDTH 400
#define RESOLUTION_HEIGHT 400

void initGL(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    gluPerspective(45.0f, (GLfloat)RESOLUTION_WIDTH / RESOLUTION_HEIGHT, 0.1f, 100.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void render(void)
{
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_TRIANGLES);
        glVertex3f(0.0f, 0.0f, 0.0f);
        glVertex3f(1.0f, 0.0f, 0.0f);
        glVertex3f(0.5f, 0.8660254f, 0.0f);
    glEnd();

    glutSwapBuffers();
}

int main(int argc, char** argv)
{
    SDL_Init(SDL_INIT_VIDEO);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_OPENGL);
    SDL_GL_SetAttribute(SDL_GL_DEPTH_SIZE, 24);
    SDL_GL_SetAttribute(SDL_GL_STENCIL_SIZE, 8);
    SDL_GL_SetAttribute(SDL_GL_DOUBLEBUFFER, 1);
    SDL_GL_SetAttribute(SDL_GL_SWAP_CONTROL, 1);

    SDL_Window* window = SDL_CreateWindow(argv[1], 0, 0, RESOLUTION_WIDTH, RESOLUTION_HEIGHT, SDL_WINDOW_OPENGL);
    SDL_GLContext glContext = SDL_GL_CreateContext(window);

    SDL_SetRenderTarget(window, glContext);

    initGL();

    while(1)
    {
        SDL_Event e;
        while(SDL_PollEvent(&e))
        {
            if(e.type == SDL_QUIT)
            {
                return 0;
            }
        }

        render();
        SDL_GL_SwapBuffers();
    }
}
```





"Hello Triangle".

- GL1 uses SDL,
- GL3+ and Vulkan use GLFW3.

Admittedly the Vulkan version has more comments and the GL versions could do more error checking..

[illegible]



```

// Create the device
auto device = cudaDeviceGet (&deviceId, 0);
auto deviceProp = cudaGetDeviceProperties (&deviceProp, deviceId);
auto deviceName = cudaGetDeviceName (deviceId);
auto deviceSerial = cudaGetDeviceSerial (deviceId);

// (Optional) Print device information
if (deviceId == 0) {
    printf ("Device: %s\n", deviceName);
    printf ("Serial: %d\n", deviceSerial);
}

// Create the device context
auto context = cudaCtxGetDevice (deviceId);
auto contextProp = cudaCtxGetDeviceProperties (&contextProp, deviceId);

// Create the device memory
auto mem = cudaMemGetDevice (deviceId);
auto memProp = cudaMemGetDeviceProperties (&memProp, deviceId);

// Create the device stream
auto stream = cudaStreamCreate (0);
auto streamProp = cudaStreamGetDeviceProperties (&streamProp, deviceId);

// Create the device event
auto event = cudaEventCreate (0);
auto eventProp = cudaEventGetDeviceProperties (&eventProp, deviceId);

// Create the device kernel
auto kernel = cudaKernelGetDevice (deviceId);
auto kernelProp = cudaKernelGetDeviceProperties (&kernelProp, deviceId);

// Create the device module
auto module = cudaModuleGetDevice (deviceId);
auto moduleProp = cudaModuleGetDeviceProperties (&moduleProp, deviceId);

// Create the device program
auto program = cudaProgramGetDevice (deviceId);
auto programProp = cudaProgramGetDeviceProperties (&programProp, deviceId);

// Create the device function
auto function = cudaFunctionGetDevice (deviceId);
auto functionProp = cudaFunctionGetDeviceProperties (&functionProp, deviceId);

// Create the device variable
auto variable = cudaVariableGetDevice (deviceId);
auto variableProp = cudaVariableGetDeviceProperties (&variableProp, deviceId);

// Create the device constant
auto constant = cudaConstantGetDevice (deviceId);
auto constantProp = cudaConstantGetDeviceProperties (&constantProp, deviceId);

// Create the device texture
auto texture = cudaTextureGetDevice (deviceId);
auto textureProp = cudaTextureGetDeviceProperties (&textureProp, deviceId);

// Create the device surface
auto surface = cudaSurfaceGetDevice (deviceId);
auto surfaceProp = cudaSurfaceGetDeviceProperties (&surfaceProp, deviceId);

// Create the device image
auto image = cudaImageGetDevice (deviceId);
auto imageProp = cudaImageGetDeviceProperties (&imageProp, deviceId);

// Create the device volume
auto volume = cudaVolumeGetDevice (deviceId);
auto volumeProp = cudaVolumeGetDeviceProperties (&volumeProp, deviceId);

// Create the device vector
auto vector = cudaVectorGetDevice (deviceId);
auto vectorProp = cudaVectorGetDeviceProperties (&vectorProp, deviceId);

// Create the device matrix
auto matrix = cudaMatrixGetDevice (deviceId);
auto matrixProp = cudaMatrixGetDeviceProperties (&matrixProp, deviceId);

// Create the device tensor
auto tensor = cudaTensorGetDevice (deviceId);
auto tensorProp = cudaTensorGetDeviceProperties (&tensorProp, deviceId);

// Create the device scalar
auto scalar = cudaScalarGetDevice (deviceId);
auto scalarProp = cudaScalarGetDeviceProperties (&scalarProp, deviceId);

// Create the device pointer
auto pointer = cudaPointerGetDevice (deviceId);
auto pointerProp = cudaPointerGetDeviceProperties (&pointerProp, deviceId);

// Create the device array
auto array = cudaArrayGetDevice (deviceId);
auto arrayProp = cudaArrayGetDeviceProperties (&arrayProp, deviceId);

// Create the device vector of pointers
auto vectorOfPointers = cudaVectorOfPointersGetDevice (deviceId);
auto vectorOfPointersProp = cudaVectorOfPointersGetDeviceProperties (&vectorOfPointersProp, deviceId);

// Create the device matrix of pointers
auto matrixOfPointers = cudaMatrixOfPointersGetDevice (deviceId);
auto matrixOfPointersProp = cudaMatrixOfPointersGetDeviceProperties (&matrixOfPointersProp, deviceId);

// Create the device tensor of pointers
auto tensorOfPointers = cudaTensorOfPointersGetDevice (deviceId);
auto tensorOfPointersProp = cudaTensorOfPointersGetDeviceProperties (&tensorOfPointersProp, deviceId);

// Create the device scalar of pointers
auto scalarOfPointers = cudaScalarOfPointersGetDevice (deviceId);
auto scalarOfPointersProp = cudaScalarOfPointersGetDeviceProperties (&scalarOfPointersProp, deviceId);

// Create the device pointer of pointers
auto pointerOfPointers = cudaPointerOfPointersGetDevice (deviceId);
auto pointerOfPointersProp = cudaPointerOfPointersGetDeviceProperties (&pointerOfPointersProp, deviceId);

// Create the device array of pointers
auto arrayOfPointers = cudaArrayOfPointersGetDevice (deviceId);
auto arrayOfPointersProp = cudaArrayOfPointersGetDeviceProperties (&arrayOfPointersProp, deviceId);

// Create the device vector of arrays
auto vectorOfArrays = cudaVectorOfArraysGetDevice (deviceId);
auto vectorOfArraysProp = cudaVectorOfArraysGetDeviceProperties (&vectorOfArraysProp, deviceId);

// Create the device matrix of arrays
auto matrixOfArrays = cudaMatrixOfArraysGetDevice (deviceId);
auto matrixOfArraysProp = cudaMatrixOfArraysGetDeviceProperties (&matrixOfArraysProp, deviceId);

// Create the device tensor of arrays
auto tensorOfArrays = cudaTensorOfArraysGetDevice (deviceId);
auto tensorOfArraysProp = cudaTensorOfArraysGetDeviceProperties (&tensorOfArraysProp, deviceId);

// Create the device scalar of arrays
auto scalarOfArrays = cudaScalarOfArraysGetDevice (deviceId);
auto scalarOfArraysProp = cudaScalarOfArraysGetDeviceProperties (&scalarOfArraysProp, deviceId);

// Create the device pointer of arrays
auto pointerOfArrays = cudaPointerOfArraysGetDevice (deviceId);
auto pointerOfArraysProp = cudaPointerOfArraysGetDeviceProperties (&pointerOfArraysProp, deviceId);

// Create the device array of arrays
auto arrayOfArrays = cudaArrayOfArraysGetDevice (deviceId);
auto arrayOfArraysProp = cudaArrayOfArraysGetDeviceProperties (&arrayOfArraysProp, deviceId);

// Create the device vector of volumes
auto vectorOfVolumes = cudaVectorOfVolumesGetDevice (deviceId);
auto vectorOfVolumesProp = cudaVectorOfVolumesGetDeviceProperties (&vectorOfVolumesProp, deviceId);

// Create the device matrix of volumes
auto matrixOfVolumes = cudaMatrixOfVolumesGetDevice (deviceId);
auto matrixOfVolumesProp = cudaMatrixOfVolumesGetDeviceProperties (&matrixOfVolumesProp, deviceId);

// Create the device tensor of volumes
auto tensorOfVolumes = cudaTensorOfVolumesGetDevice (deviceId);
auto tensorOfVolumesProp = cudaTensorOfVolumesGetDeviceProperties (&tensorOfVolumesProp, deviceId);

// Create the device scalar of volumes
auto scalarOfVolumes = cudaScalarOfVolumesGetDevice (deviceId);
auto scalarOfVolumesProp = cudaScalarOfVolumesGetDeviceProperties (&scalarOfVolumesProp, deviceId);

// Create the device pointer of volumes
auto pointerOfVolumes = cudaPointerOfVolumesGetDevice (deviceId);
auto pointerOfVolumesProp = cudaPointerOfVolumesGetDeviceProperties (&pointerOfVolumesProp, deviceId);

// Create the device array of volumes
auto arrayOfVolumes = cudaArrayOfVolumesGetDevice (deviceId);
auto arrayOfVolumesProp = cudaArrayOfVolumesGetDeviceProperties (&arrayOfVolumesProp, deviceId);

// Create the device vector of matrices
auto vectorOfMatrices = cudaVectorOfMatricesGetDevice (deviceId);
auto vectorOfMatricesProp = cudaVectorOfMatricesGetDeviceProperties (&vectorOfMatricesProp, deviceId);

// Create the device matrix of matrices
auto matrixOfMatrices = cudaMatrixOfMatricesGetDevice (deviceId);
auto matrixOfMatricesProp = cudaMatrixOfMatricesGetDeviceProperties (&matrixOfMatricesProp, deviceId);

// Create the device tensor of matrices
auto tensorOfMatrices = cudaTensorOfMatricesGetDevice (deviceId);
auto tensorOfMatricesProp = cudaTensorOfMatricesGetDeviceProperties (&tensorOfMatricesProp, deviceId);

// Create the device scalar of matrices
auto scalarOfMatrices = cudaScalarOfMatricesGetDevice (deviceId);
auto scalarOfMatricesProp = cudaScalarOfMatricesGetDeviceProperties (&scalarOfMatricesProp, deviceId);

// Create the device pointer of matrices
auto pointerOfMatrices = cudaPointerOfMatricesGetDevice (deviceId);
auto pointerOfMatricesProp = cudaPointerOfMatricesGetDeviceProperties (&pointerOfMatricesProp, deviceId);

// Create the device array of matrices
auto arrayOfMatrices = cudaArrayOfMatricesGetDevice (deviceId);
auto arrayOfMatricesProp = cudaArrayOfMatricesGetDeviceProperties (&arrayOfMatricesProp, deviceId);

// Create the device vector of tensors
auto vectorOfTensors = cudaVectorOfTensorsGetDevice (deviceId);
auto vectorOfTensorsProp = cudaVectorOfTensorsGetDeviceProperties (&vectorOfTensorsProp, deviceId);

// Create the device matrix of tensors
auto matrixOfTensors = cudaMatrixOfTensorsGetDevice (deviceId);
auto matrixOfTensorsProp = cudaMatrixOfTensorsGetDeviceProperties (&matrixOfTensorsProp, deviceId);

// Create the device tensor of tensors
auto tensorOfTensors = cudaTensorOfTensorsGetDevice (deviceId);
auto tensorOfTensorsProp = cudaTensorOfTensorsGetDeviceProperties (&tensorOfTensorsProp, deviceId);

// Create the device scalar of tensors
auto scalarOfTensors = cudaScalarOfTensorsGetDevice (deviceId);
auto scalarOfTensorsProp = cudaScalarOfTensorsGetDeviceProperties (&scalarOfTensorsProp, deviceId);

// Create the device pointer of tensors
auto pointerOfTensors = cudaPointerOfTensorsGetDevice (deviceId);
auto pointerOfTensorsProp = cudaPointerOfTensorsGetDeviceProperties (&pointerOfTensorsProp, deviceId);

// Create the device array of tensors
auto arrayOfTensors = cudaArrayOfTensorsGetDevice (deviceId);
auto arrayOfTensorsProp = cudaArrayOfTensorsGetDeviceProperties (&arrayOfTensorsProp, deviceId);

// Create the device vector of scalars
auto vectorOfScalars = cudaVectorOfScalarsGetDevice (deviceId);
auto vectorOfScalarsProp = cudaVectorOfScalarsGetDeviceProperties (&vectorOfScalarsProp, deviceId);

// Create the device matrix of scalars
auto matrixOfScalars = cudaMatrixOfScalarsGetDevice (deviceId);
auto matrixOfScalarsProp = cudaMatrixOfScalarsGetDeviceProperties (&matrixOfScalarsProp, deviceId);

// Create the device tensor of scalars
auto tensorOfScalars = cudaTensorOfScalarsGetDevice (deviceId);
auto tensorOfScalarsProp = cudaTensorOfScalarsGetDeviceProperties (&tensorOfScalarsProp, deviceId);

// Create the device scalar of scalars
auto scalarOfScalars = cudaScalarOfScalarsGetDevice (deviceId);
auto scalarOfScalarsProp = cudaScalarOfScalarsGetDeviceProperties (&scalarOfScalarsProp, deviceId);

// Create the device pointer of scalars
auto pointerOfScalars = cudaPointerOfScalarsGetDevice (deviceId);
auto pointerOfScalarsProp = cudaPointerOfScalarsGetDeviceProperties (&pointerOfScalarsProp, deviceId);

// Create the device array of scalars
auto arrayOfScalars = cudaArrayOfScalarsGetDevice (deviceId);
auto arrayOfScalarsProp = cudaArrayOfScalarsGetDeviceProperties (&arrayOfScalarsProp, deviceId);

// Create the device vector of pointers
auto vectorOfPointers2 = cudaVectorOfPointers2GetDevice (deviceId);
auto vectorOfPointers2Prop = cudaVectorOfPointers2GetDeviceProperties (&vectorOfPointers2Prop, deviceId);

// Create the device matrix of pointers
auto matrixOfPointers2 = cudaMatrixOfPointers2GetDevice (deviceId);
auto matrixOfPointers2Prop = cudaMatrixOfPointers2GetDeviceProperties (&matrixOfPointers2Prop, deviceId);

// Create the device tensor of pointers
auto tensorOfPointers2 = cudaTensorOfPointers2GetDevice (deviceId);
auto tensorOfPointers2Prop = cudaTensorOfPointers2GetDeviceProperties (&tensorOfPointers2Prop, deviceId);

// Create the device scalar of pointers
auto scalarOfPointers2 = cudaScalarOfPointers2GetDevice (deviceId);
auto scalarOfPointers2Prop = cudaScalarOfPointers2GetDeviceProperties (&scalarOfPointers2Prop, deviceId);

// Create the device pointer of pointers
auto pointerOfPointers2 = cudaPointerOfPointers2GetDevice (deviceId);
auto pointerOfPointers2Prop = cudaPointerOfPointers2GetDeviceProperties (&pointerOfPointers2Prop, deviceId);

// Create the device array of pointers
auto arrayOfPointers2 = cudaArrayOfPointers2GetDevice (deviceId);
auto arrayOfPointers2Prop = cudaArrayOfPointers2GetDeviceProperties (&arrayOfPointers2Prop, deviceId);

// Create the device vector of arrays
auto vectorOfArrays2 = cudaVectorOfArrays2GetDevice (deviceId);
auto vectorOfArrays2Prop = cudaVectorOfArrays2GetDeviceProperties (&vectorOfArrays2Prop, deviceId);

// Create the device matrix of arrays
auto matrixOfArrays2 = cudaMatrixOfArrays2GetDevice (deviceId);
auto matrixOfArrays2Prop = cudaMatrixOfArrays2GetDeviceProperties (&matrixOfArrays2Prop, deviceId);

// Create the device tensor of arrays
auto tensorOfArrays2 = cudaTensorOfArrays2GetDevice (deviceId);
auto tensorOfArrays2Prop = cudaTensorOfArrays2GetDeviceProperties (&tensorOfArrays2Prop, deviceId);

// Create the device scalar of arrays
auto scalarOfArrays2 = cudaScalarOfArrays2GetDevice (deviceId);
auto scalarOfArrays2Prop = cudaScalarOfArrays2GetDeviceProperties (&scalarOfArrays2Prop, deviceId);

// Create the device pointer of arrays
auto pointerOfArrays2 = cudaPointerOfArrays2GetDevice (deviceId);
auto pointerOfArrays2Prop = cudaPointerOfArrays2GetDeviceProperties (&pointerOfArrays2Prop, deviceId);

// Create the device array of arrays
auto arrayOfArrays2 = cudaArrayOfArrays2GetDevice (deviceId);
auto arrayOfArrays2Prop = cudaArrayOfArrays2GetDeviceProperties (&arrayOfArrays2Prop, deviceId);

// Create the device vector of volumes
auto vectorOfVolumes2 = cudaVectorOfVolumes2GetDevice (deviceId);
auto vectorOfVolumes2Prop = cudaVectorOfVolumes2GetDeviceProperties (&vectorOfVolumes2Prop, deviceId);

// Create the device matrix of volumes
auto matrixOfVolumes2 = cudaMatrixOfVolumes2GetDevice (deviceId);
auto matrixOfVolumes2Prop = cudaMatrixOfVolumes2GetDeviceProperties (&matrixOfVolumes2Prop, deviceId);

// Create the device tensor of volumes
auto tensorOfVolumes2 = cudaTensorOfVolumes2GetDevice (deviceId);
auto tensorOfVolumes2Prop = cudaTensorOfVolumes2GetDeviceProperties (&tensorOfVolumes2Prop, deviceId);

// Create the device scalar of volumes
auto scalarOfVolumes2 = cudaScalarOfVolumes2GetDevice (deviceId);
auto scalarOfVolumes2Prop = cudaScalarOfVolumes2GetDeviceProperties (&scalarOfVolumes2Prop, deviceId);

// Create the device pointer of volumes
auto pointerOfVolumes2 = cudaPointerOfVolumes2GetDevice (deviceId);
auto pointerOfVolumes2Prop = cudaPointerOfVolumes2GetDeviceProperties (&pointerOfVolumes2Prop, deviceId);

// Create the device array of volumes
auto arrayOfVolumes2 = cudaArrayOfVolumes2GetDevice (deviceId);
auto arrayOfVolumes2Prop = cudaArrayOfVolumes2GetDeviceProperties (&arrayOfVolumes2Prop, deviceId);

// Create the device vector of matrices
auto vectorOfMatrices2 = cudaVectorOfMatrices2GetDevice (deviceId);
auto vectorOfMatrices2Prop = cudaVectorOfMatrices2GetDeviceProperties (&vectorOfMatrices2Prop, deviceId);

// Create the device matrix of matrices
auto matrixOfMatrices2 = cudaMatrixOfMatrices2GetDevice (deviceId);
auto matrixOfMatrices2Prop = cudaMatrixOfMatrices2GetDeviceProperties (&matrixOfMatrices2Prop, deviceId);

// Create the device tensor of matrices
auto tensorOfMatrices2 = cudaTensorOfMatrices2GetDevice (deviceId);
auto tensorOfMatrices2Prop = cudaTensorOfMatrices2GetDeviceProperties (&tensorOfMatrices2Prop, deviceId);

// Create the device scalar of matrices
auto scalarOfMatrices2 = cudaScalarOfMatrices2GetDevice (deviceId);
auto scalarOfMatrices2Prop = cudaScalarOfMatrices2GetDeviceProperties (&scalarOfMatrices2Prop, deviceId);

// Create the device pointer of matrices
auto pointerOfMatrices2 = cudaPointerOfMatrices2GetDevice (deviceId);
auto pointerOfMatrices2Prop = cudaPointerOfMatrices2GetDeviceProperties (&pointerOfMatrices2Prop, deviceId);

// Create the device array of matrices
auto arrayOfMatrices2 = cudaArrayOfMatrices2GetDevice (deviceId);
auto arrayOfMatrices2Prop = cudaArrayOfMatrices2GetDeviceProperties (&arrayOfMatrices2Prop, deviceId);

// Create the device vector of tensors
auto vectorOfTensors2 = cudaVectorOfTensors2GetDevice (deviceId);
auto vectorOfTensors2Prop = cudaVectorOfTensors2GetDeviceProperties (&vectorOfTensors2Prop, deviceId);

// Create the device matrix of tensors
auto matrixOfTensors2 = cudaMatrixOfTensors2GetDevice (deviceId);
auto matrixOfTensors2Prop = cudaMatrixOfTensors2GetDeviceProperties (&matrixOfTensors2Prop, deviceId);

// Create the device tensor of tensors
auto tensorOfTensors2 = cudaTensorOfTensors2GetDevice (deviceId);
auto tensorOfTensors2Prop = cudaTensorOfTensors2GetDeviceProperties (&tensorOfTensors2Prop, deviceId);

// Create the device scalar of tensors
auto scalarOfTensors2 = cudaScalarOfTensors2GetDevice (deviceId);
auto scalarOfTensors2Prop = cudaScalarOfTensors2GetDeviceProperties (&scalarOfTensors2Prop, deviceId);

// Create the
```



[illegible]

[illegible]

OpenGL 1.x

OpenGL 3+

Vulkan

"Hello Triangle".
- GL1 uses SDL,
- GL3+ and Vulkan
use GLFW3.

Admittedly the Vulkan
version has more comments
and the GL versions could
do more error checking.



```
VkResult res = vkAcquireNextImageKHR(device, swapchain, UINT32_MAX, imageAvailableSemaphore, VK_NULL_HANDLE, &imageIndex);

if (res == VK_ERROR_OUT_OF_DATE_KHR) {
    vkDeviceWaitIdle(device);
    return VK_ERROR_OUT_OF_DATE_KHR;
} else if (res != VK_SUCCESS) {
    return res;
}

// Wait for image to be available and draw
VkSubmitInfo submitInfo = {};
submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;
submitInfo.commandBuffers = &commandBuffer;
submitInfo.queueIndex = 0;
submitInfo.waitSemaphoreCount = 1;
submitInfo.pWaitSemaphores = &imageAvailableSemaphore;
submitInfo.signalSemaphoreCount = 1;
submitInfo.pSignalSemaphores = &imageReadySemaphore;

// This is the stage where the queue should wait on the semaphore
VkPipelineStageFlags waitStageFlags = VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT;
submitInfo.pWaitStageFlags = &waitStageFlags;

submitInfo.commandBufferCount = 1;
submitInfo.pCommandBuffers = &commandBuffer;

if (vkQueueSubmit(queue, 1, submitInfo, VK_NULL_HANDLE) != VK_SUCCESS) {
    return VK_ERROR_OUT_OF_DATE_KHR;
}

// Present draw image
// Note: semaphore here is not strictly necessary, because commands are processed in submission order within a single queue
VkPresentInfoKHR presentInfo = {};
presentInfo.sType = VK_STRUCTURE_TYPE_PRESENT_INFO_KHR;
presentInfo.queueIndex = 0;
presentInfo.pPresentImageIndices = &imageIndex;
presentInfo.pSemaphores = &imageReadySemaphore;

presentInfo.swapchainCount = 1;
presentInfo.pSwapchains = &swapchain;

res = vkQueuePresentKHR(queue, &presentInfo);

if (res == VK_ERROR_OUT_OF_DATE_KHR || res == VK_ERROR_SWAPCHAIN_KHR || VK_SUCCESS) {
    vkDeviceWaitIdle(device);
    return VK_ERROR_OUT_OF_DATE_KHR;
} else if (res != VK_SUCCESS) {
    return res;
}

return VK_SUCCESS;
}

int main() {
    TriangleApplication app;
    app.run();
    return 0;
}
```


Brief History of Vulkan



K H R O N O S[®]
G R O U P

Brief History of Vulkan

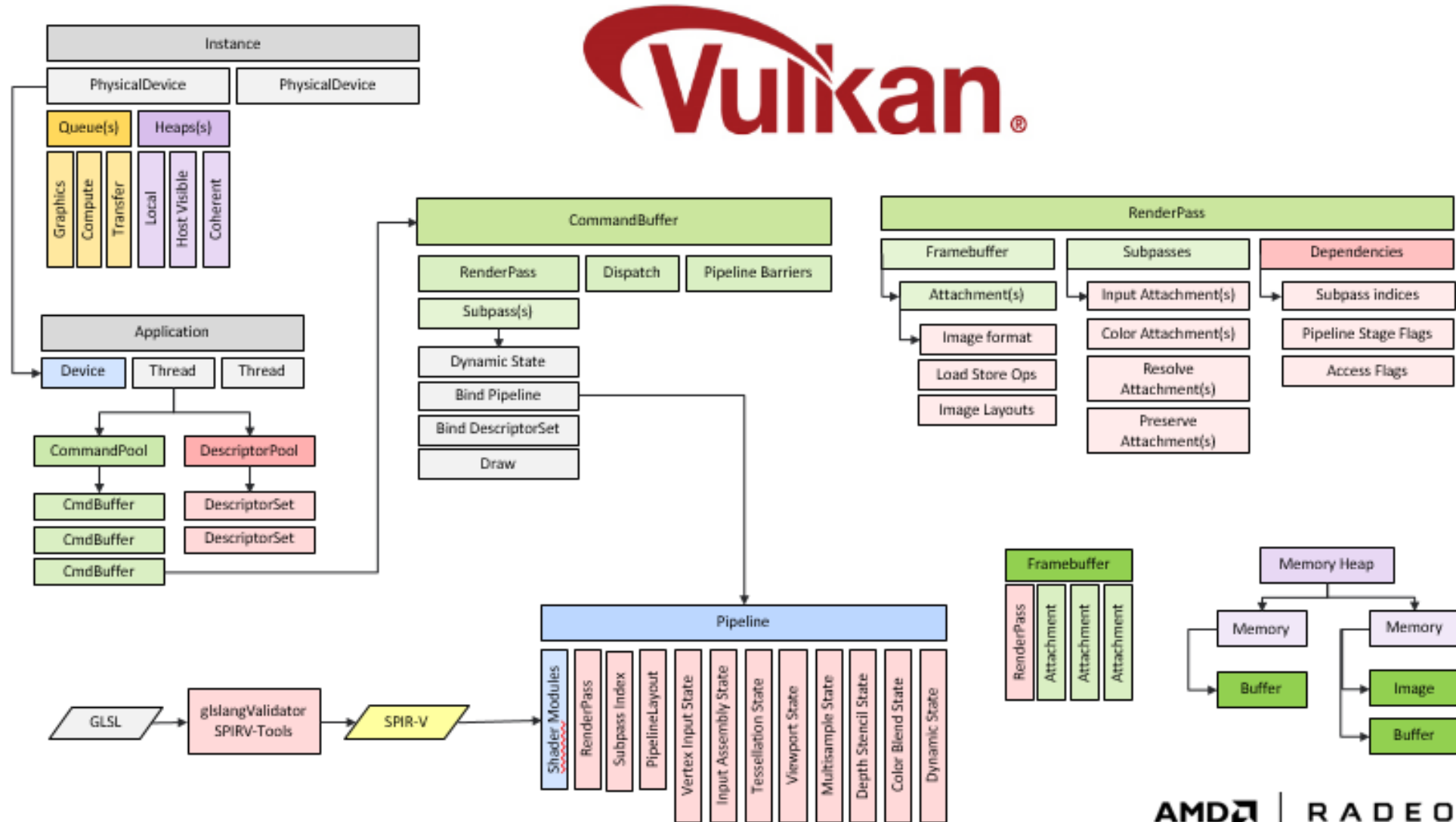


- **2015** – Vulkan 1.0 draft announced (derived from Mantle API)
- **2016** – full specification + open-sourced SDK
- **2018** – Vulkan 1.1 released
 - HLSL, YCbCr, Raytracing, ...
- **2020** – Vulkan 1.2 released
 - 23 proven extensions integrated: timeline semaphores, descriptor indexing, ...
- **2022** – Vulkan 1.3 released
 - 23 proven extensions integrated: dynamic rendering, dynamic state, improved synchronization, ...

Why is Vulkan Faster?

- Very explicit/state known in advance
 - Driver doesn't need to guess that much
- Native multithreading support
- Direct control over memory
- Synchronization

Vulkan components



Struct Based API



C API

```
typedef struct VkInstanceCreateInfo {  
    VkStructureType          sType;  
    const void*              pNext;  
    VkInstanceCreateFlags    flags;  
    const VkApplicationInfo* pApplicationInfo;  
    uint32_t                 enabledLayerCount;  
    const char* const*       ppEnabledLayerNames;  
    uint32_t                 enabledExtensionCount;  
    const char* const*       ppEnabledExtensionNames;  
} VkInstanceCreateInfo;  
  
VkResult vkCreateInstance(  
    const VkInstanceCreateInfo* pCreateInfo,  
    const VkAllocationCallbacks* pAllocator,  
    VkInstance* pInstance  
);
```

Struct Based API



C++ API (wrapper)

- Designated initializers:
- Setters:
- Constructor
- C-like fill

```
// Designated initializer
vk::InstanceCreateInfo{
    .flags = ...,
    .pApplicationInfo = ...,
    ...
};
```

```
// Setters
vk::InstanceCreateInfo()
    .setFlags(...)
    .setPApplicationInfo(...)
    ...
;
```

```
template <typename Dispatch>
typename ResultValueType<vk::Instance>::type createInstanceUnique(
    const vk::InstanceCreateInfo& createInfo,
    Optional<const vk::AllocationCallbacks> allocator,
    Dispatch const& d
);
```

Instance



- Application state
- Typically, single instance for an application
- Instances **cannot** share resources
- Libraries usually have their own instance
- Extension needed for drawing:
`VK_KHR_surface`
- Validation layers:
`VK_LAYER_KHRONOS_validation`

```
auto appCI = vk::ApplicationInfo()  
    .setApiVersion(VK_API_VERSION_1_3)  
    .setPApplicationName(mApplicationName);  
  
const auto instanceCI = vk::InstanceCreateInfo()  
    .setPApplicationInfo(&appCI)  
    .setPEnabledLayerNames(layers)  
    .setPEnabledExtensionNames(kInstanceExtensions);  
  
mInstance = createInstanceUnique(instanceCI);
```

Surface



- Surface usually created by 3rd party window library
 - SDL
 - GLFW
 - WinAPI
 - ...

```
VkSurfaceKHR surface;  
SDL_Vulkan_CreateSurface(  
    mWindow,  
    mInstance.get(),  
    &surface  
);
```

Physical Device



- Picking actual device/GPU for rendering/work

```
// Query devices
auto devices = mInstance->enumeratePhysicalDevices();

// Loop through devices, and pick best one
for (auto& device : devices)
{
    // Check device capabilities
    ...

    if (device_satisfied)
    {
        mPhysicalDevice = device;
        break;
    }
}
```

Device (logical device)



- On top of Physical device create (logical) device
- Pick execution queues
- Enabling device extensions

```
auto queuePriorities = { 1.f };
auto queueCIs = {
    vk::DeviceQueueCreateInfo()
        .setQueueFamilyIndex(mGeneralQueueFamilyIndex)
        .setQueueCount(1)
        .setQueuePriorities(queuePriorities)
};

const auto deviceCI = vk::DeviceCreateInfo()
    .setQueueCreateInfos(queueCIs)
    .setPEnabledLayerNames(layers)
    .setPEnabledExtensionNames(deviceExtensions);

mDevice = mPhysicalDevice.createDeviceUnique(deviceCI);

// Extract queues from device
mGeneralQueue = mDevice->getQueue(
    mGeneralQueueFamilyIndex,
    0
);
```

Swapchain

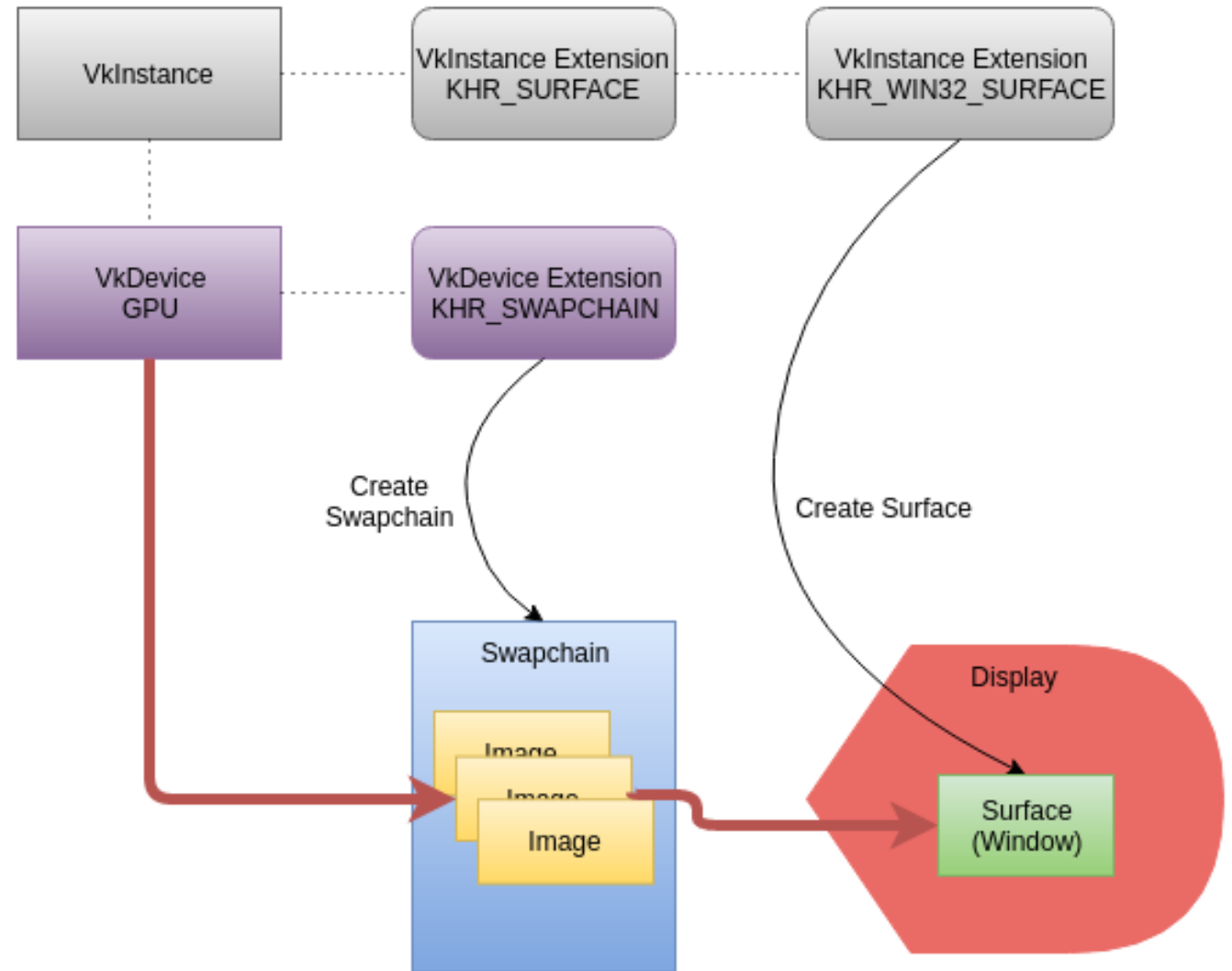


- Collection of images
 - There is **no implicit** framebuffer!
- Used for rendering/presenting
- Presentation modes
 - FIFO
 - Mailbox
 - Immediate
- Recreated on window size change

```
const auto swapchainCI = vk::SwapchainCreateInfoKHR()  
    .setImageColorSpace(vk::ColorSpaceKHR::eSrgbNonlinear)  
    .setImageExtent({ mWindowExtent.x, mWindowExtent.y })  
    .setImageFormat(swapchainImageFormat)  
    .setImageSharingMode(vk::SharingMode::eExclusive)  
    .setImageUsage(vk::ImageUsageFlagBits::eColorAttachment)  
    .setImageArrayLayers(1)  
    .setMinImageCount(kFramesInFlight)  
    .setSurface(*mSurface)  
    .setQueueFamilyIndices(families)  
    .setPresentMode(vk::PresentModeKHR::eMailbox);  
  
mSwapchain = mDevice->createSwapchainKHRUnique(swapchainCI);
```

Swapchain

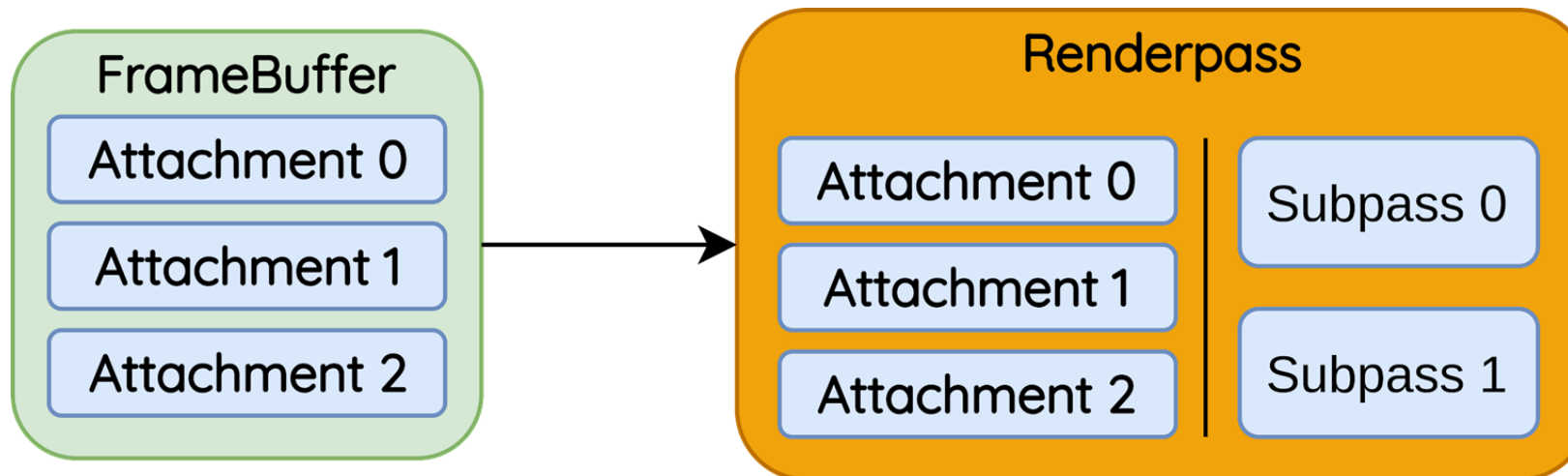
- Collection of images
 - There is **no implicit** framebuffer!
- Used for rendering/presenting
- Presentation modes
 - FIFO
 - Mailbox
 - Immediate
- Recreated on window size change



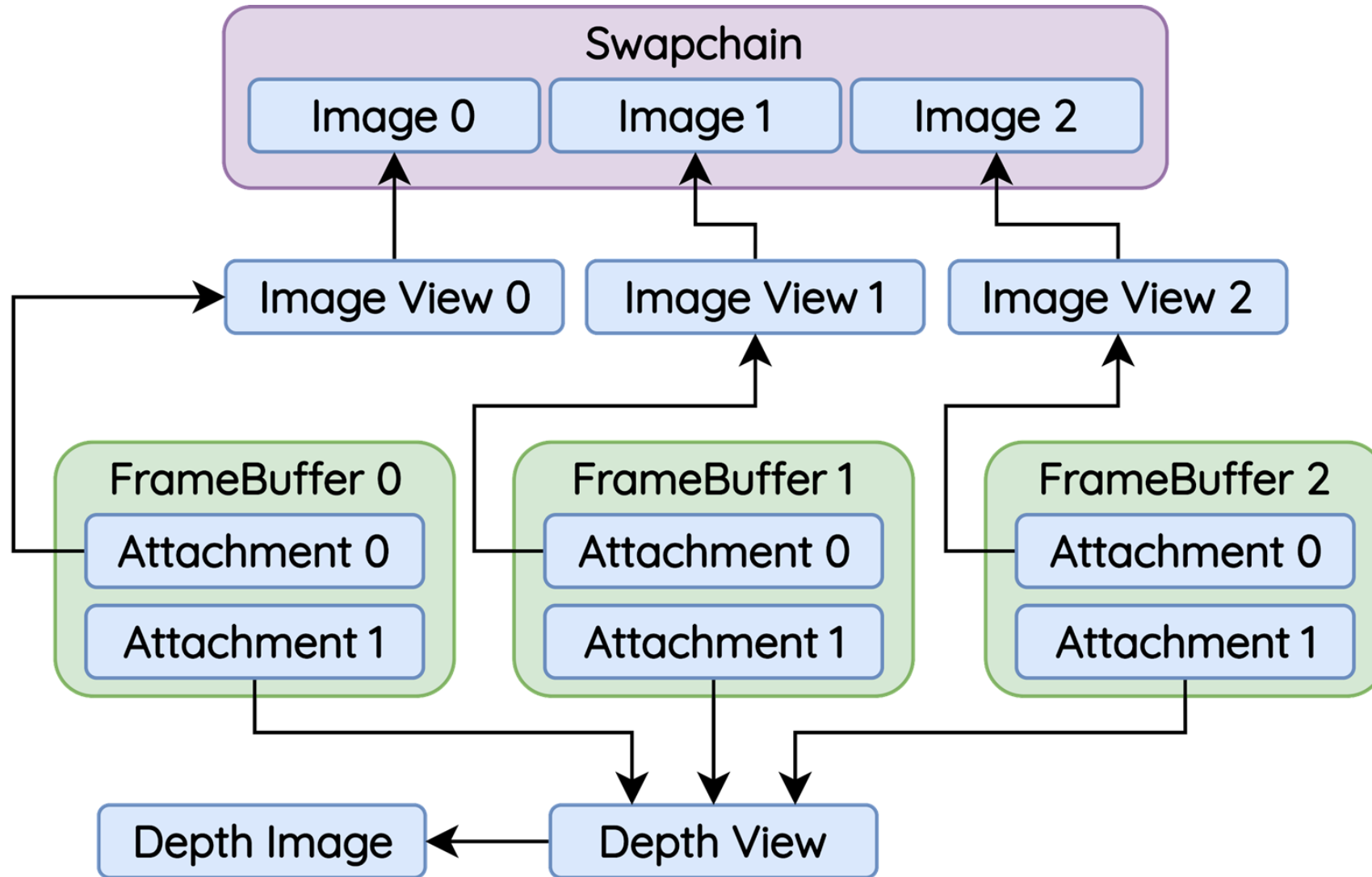
Framebuffer



```
auto framebufferCI = vk::FramebufferCreateInfo()  
    .setAttachments(attachments)  
    .setRenderPass(mRenderPass)  
    .setWidth(mSwapchainExtent.width)  
    .setHeight(mSwapchainExtent.height)  
    .setLayers(1);  
  
mFramebuffer = mDevice->createFramebufferUnique(  
    framebufferCI  
);
```



Framebuffer



Renderpass



- Vulkan concept
- Collection of images (framebuffer), subpasses and dependencies between them
- Motivation:
 - Automatic attachment load/stores
 - Automatic layout transitions
 - Dependencies without barriers
 - Subpasses -- effective for tile-based (mobile) architectures

```
auto subpass = vk::SubpassDescription()  
    .setColorAttachments(attachmentReferences)  
    .setPipelineBindPoint(vk::PipelineBindPoint::eGraphics)  
    .setPDepthStencilAttachment(depthAttachmentReference);  
  
auto renderpassCI = vk::RenderPassCreateInfo()  
    .setAttachments(attachmentDescriptions)  
    .setSubpasses(subpass)  
    .setDependencies(dependencies);  
  
mRenderpass = mDevice->createRenderPassUnique(  
    renderpassCI  
);
```

Renderpass

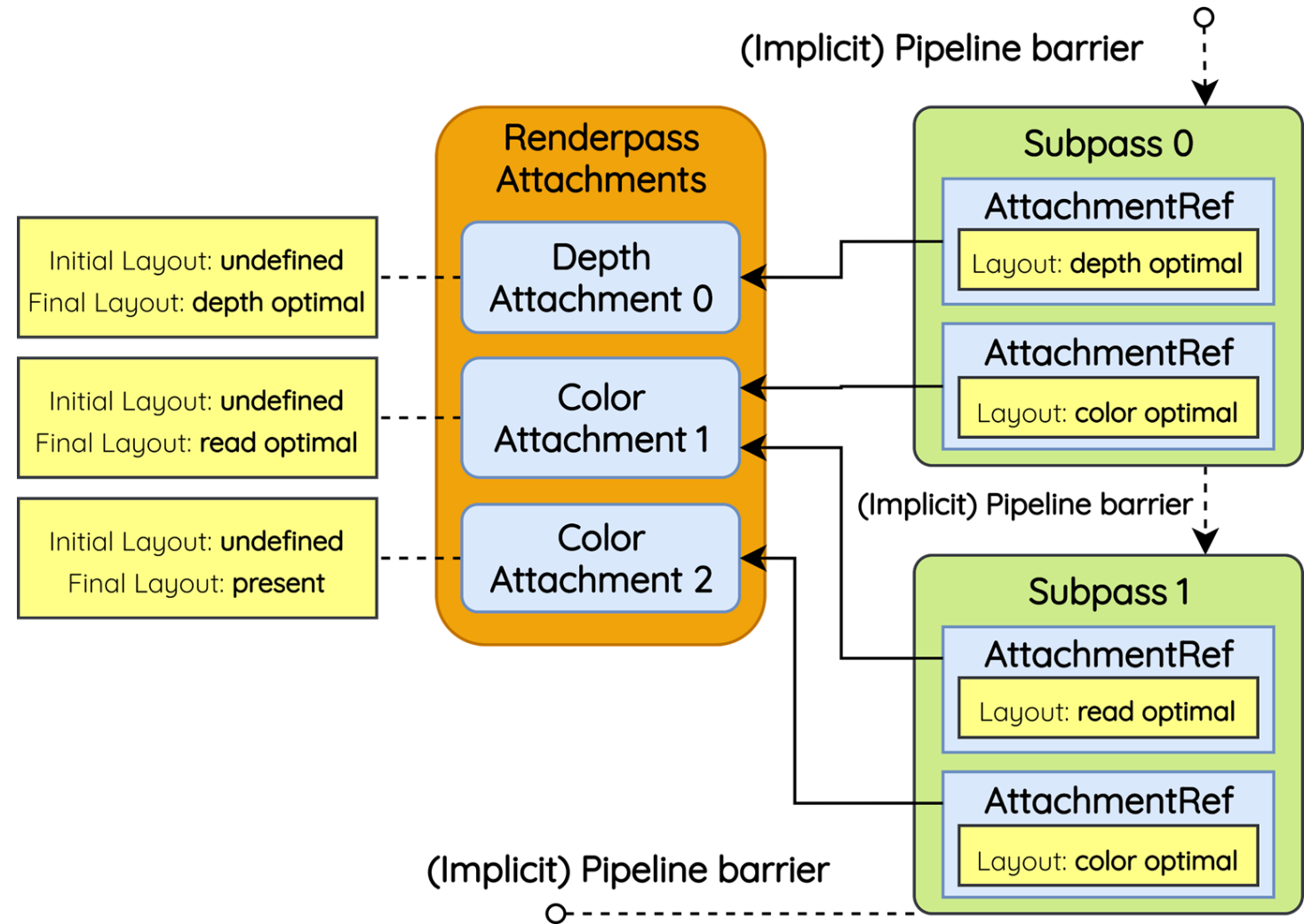


- Vulkan concept
- Collection of images (framebuffer), subpasses and dependencies between them
- Motivation:
 - Automatic attachment load/stores
 - Automatic layout transitions
 - Dependencies without barriers
 - Subpasses -- effective for tile-based (mobile) architectures

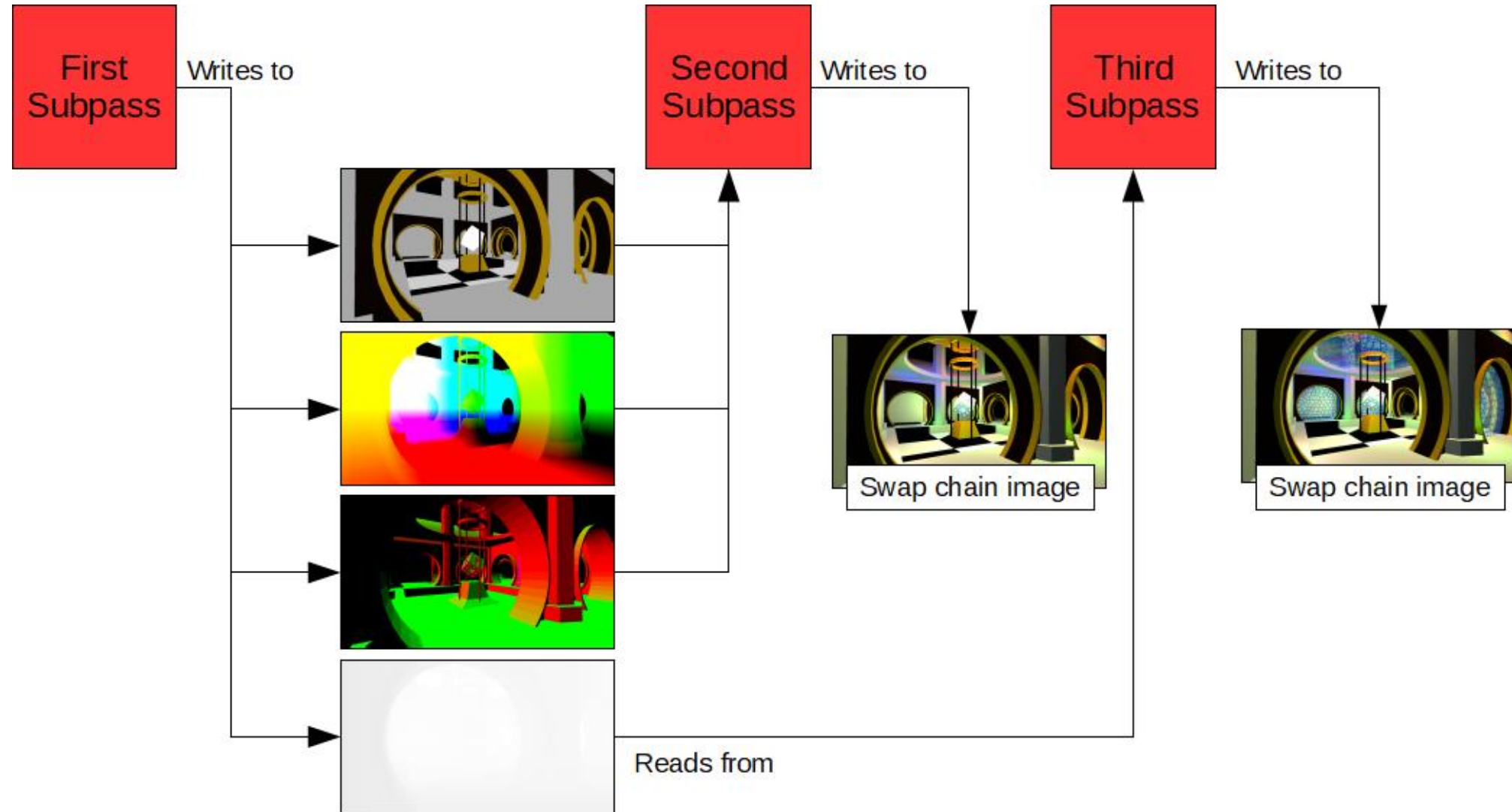
```
auto renderpassI = vk::RenderPassBeginInfo()  
    .setRenderPass(*mRenderpass)  
    .setFramebuffer(*mFramebuffer)  
    .setRenderArea(mSwapchainExtent)  
    .setClearValues(clearValues);  
  
cmd.beginRenderPass(  
    renderpassI,  
    vk::SubpassContents::eInline  
);
```

Subpass

- Phase of rendering, in which we work with subset of Renderpass attachments
- Every Renderpass needs to have at least 1 Subpass
- n-Subpasses/n-Renderpasses
 - Use of multiple Subpasses is optional



Deferred Shading with Subpasses



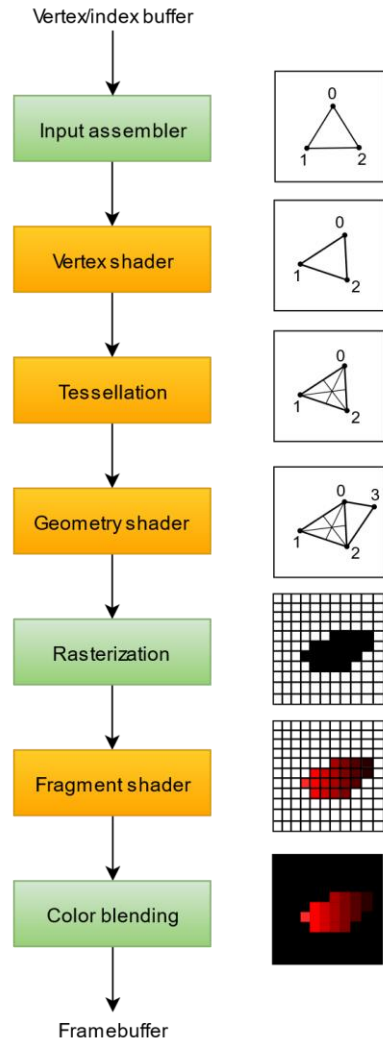
Pipeline Objects



- Holds rendering state
- Immutable
 - Everything is fixed
 - Except some dynamic states
- Shader modules
- Reference to **RenderPass** and index of starting **Subpass**

```
auto layoutCI = vk::PipelineLayoutCreateInfo()  
    .setSetLayouts(setLayouts)  
    .setPushConstantRanges(ranges);  
  
mLayout = mDevice->createPipelineLayoutUnique(layoutCI)
```

Pipeline Objects



```
auto pipelineCI = vk::GraphicsPipelineCreateInfo()  
    .setStages(shaderStages)  
    .setPVertexInputState(&vertexInputInfo)  
    .setPInputAssemblyState(&inputAssemblyInfo)  
    .setPViewportState(&viewportInfo)  
    .setPRasterizationState(&rasterizer)  
    .setPMultisampleState(&multisampling)  
    .setPDepthStencilState(&depthStencil)  
    .setPColorBlendState(&blendingInfo)  
    .setLayout(*mPipelineLayout)  
    .setRenderPass(*mRenderPass)  
    .setSubpass(0)  
    .setBasePipelineHandle(nullptr) // not deriving  
    .setBasePipelineIndex(-1); // from existing pipeline
```

```
mPipeline = mDevice->createGraphicsPipelineUnique(  
    mPipelineCache,  
    pipelineCI  
);
```


Descriptor Sets



- Vulkan way of organizing shader input data
- Input data are organized to groups (descriptor sets)
- Only references (pointers) to actual data
- Allocated from **DescriptorPool**

```
auto layoutBindings = {  
    vk::DescriptorSetLayoutBinding()  
        .setBinding(0)  
        .setDescriptorCount(1)  
        .setDescriptorType(vk::DescriptorType::eCombinedImageSampler)  
        .setStageFlags(vk::ShaderStageFlagBits::eFragment),  
  
    ...  
};  
  
auto layoutCI = vk::DescriptorSetLayoutCreateInfo()  
    .setBindings(layoutBindings);  
  
mDescriptorSetLayout = mDevice->createDescriptorSetLayoutUnique(  
    layoutCI  
);
```

Descriptor Sets



- Vulkan way of organizing shader input data
- Input data are organized to groups (descriptor sets)
- Only references (pointers) to actual data
- Allocated from **DescriptorPool**

```
auto allocI = vk::DescriptorSetAllocateInfo()  
    .setDescriptorPool(mDescriptorPool)  
    .setSetLayouts(*mDescriptorSetLayout);  
  
mDescriptorSet = mDevice->allocateDescriptorSetsUnique(allocI);
```

Descriptor Sets



- Vulkan way of organizing shader input data
- Input data are organized to groups (descriptor sets)
- Only references (pointers) to actual data
- Allocated from **DescriptorPool**

```
auto writeSet = {  
    vk::WriteDescriptorSet()  
        .setDstSet(mDescriptorSet)  
        .setDstBinding(0)  
        .setDescriptorCount(1)  
        .setDescriptorType(vk::DescriptorType::eCombinedImageSampler)  
        .setImageInfo({ ... } ),  
};  
  
mDevice->updateDescriptorSets(writeSet, {});
```

Descriptor Sets



- Vulkan way of organizing shader input data
- Input data are organized to groups (descriptor sets)
- Only references (pointers) to actual data
- Allocated from **DescriptorPool**

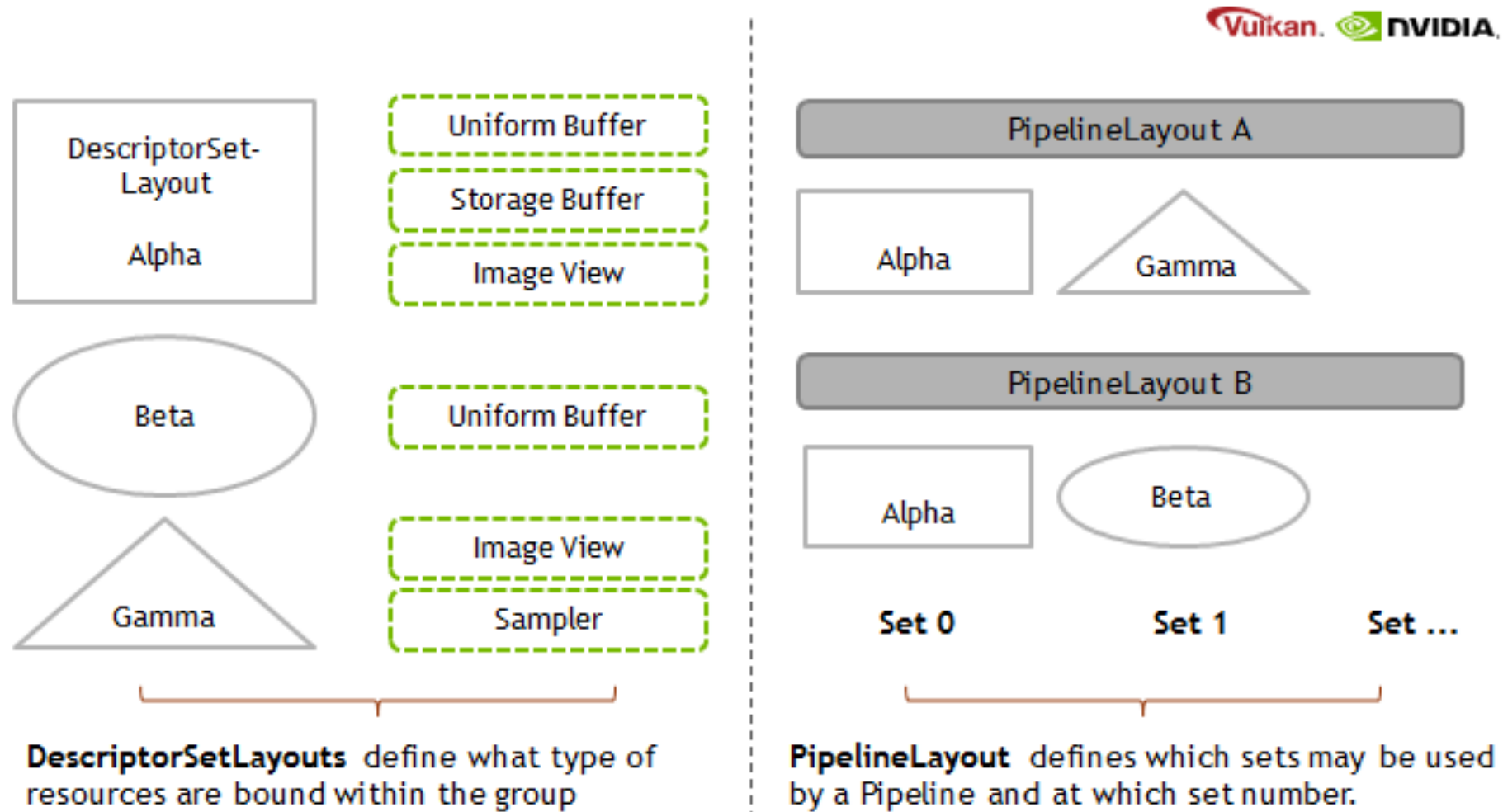
```
layout(set = 0, binding = 0) uniform sampler2D samplerAlbedo;  
layout(set = 0, binding = 1) uniform sampler2D samplerNormal;  
layout(set = 0, binding = 2) uniform sampler2D samplerEmission;  
layout(set = 0, binding = 3) uniform sampler2D samplerPBRWorkflow;
```

```
layout(set = 1, binding = 0) uniform sampler2D samplerDepth;  
layout(set = 1, binding = 1) uniform sampler2D samplerIrradiance;  
layout(set = 1, binding = 2) uniform sampler2D samplerDistance;  
layout(set = 1, binding = 3) uniform sampler2D samplerReflections;
```

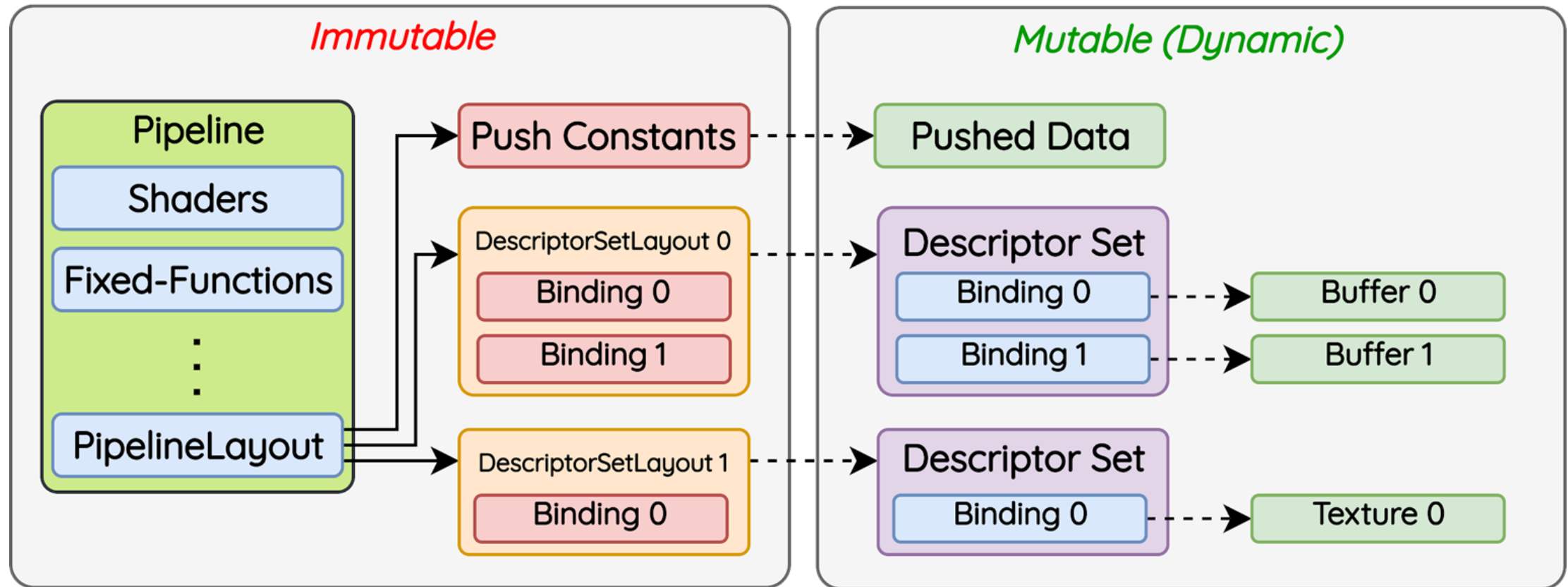
```
layout(std430, set = 2, binding = 0) uniform SceneData { ... }  
sceneUBO;
```

```
layout(std430, set = 2, binding = 1) uniform Materials { ... }  
matUBO;
```

PipelineLayout/DescriptorSetLayout



PipelineLayout/DescriptorSet



Recap

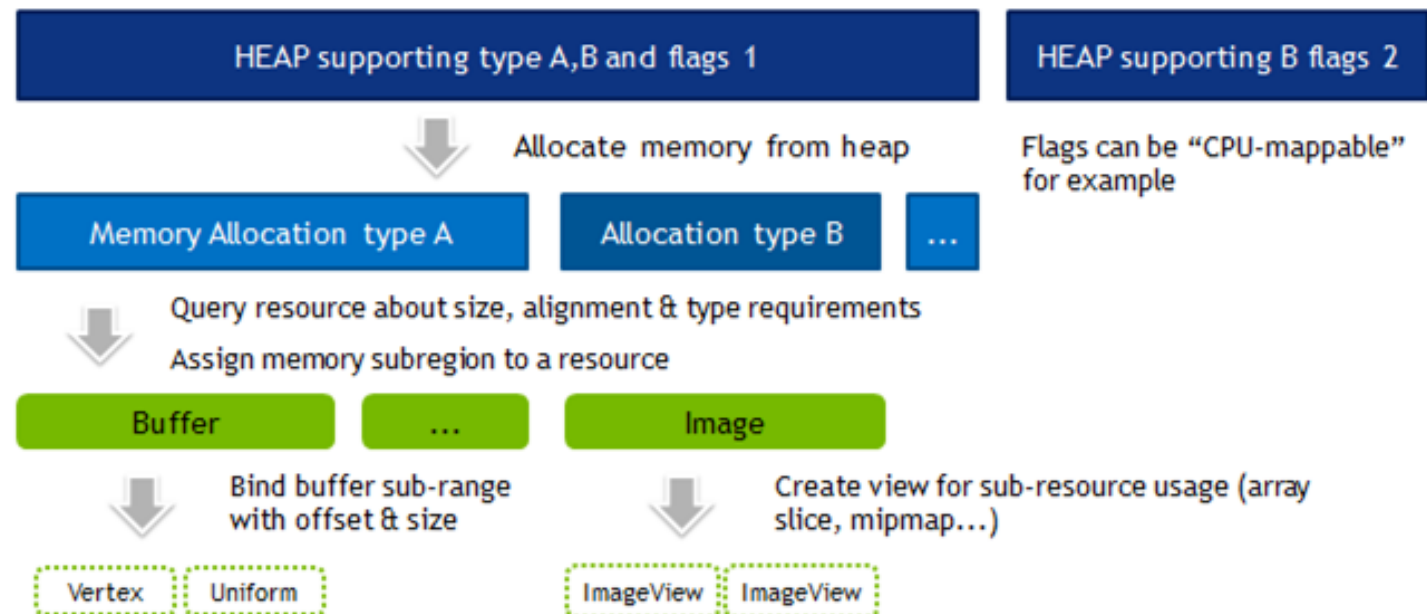


- **Swapchain**: image collection where we draw for presenting
- **Framebuffer**: set of attachments (swapchain image, color image, depth buffer, ...)
- **Renderpass**: management of Framebuffer attachments (dependencies, views, ...)
- **Pipeline**: rendering state
- **DescriptorSet**: grouping of shader resources
- **Missing**:
 - Data of models (vertices, indices, ...)
 - Command buffers
 - Synchronization primitives

Memory Allocation

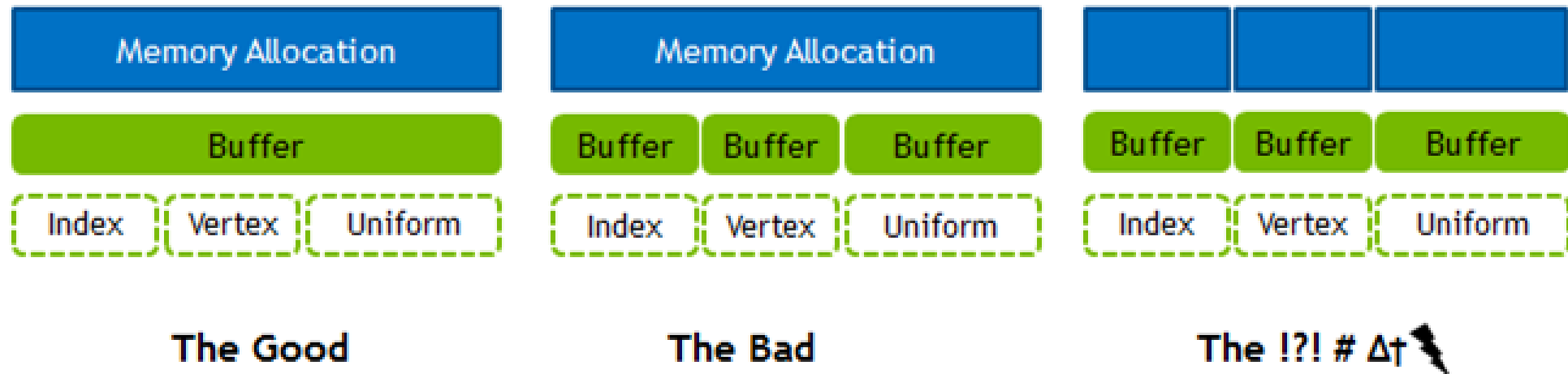


- Need to specify size and type of memory
- Before allocation of resource, we need to query resource size, alignment and type of memory
- Max limit of allocations (typically 4096)



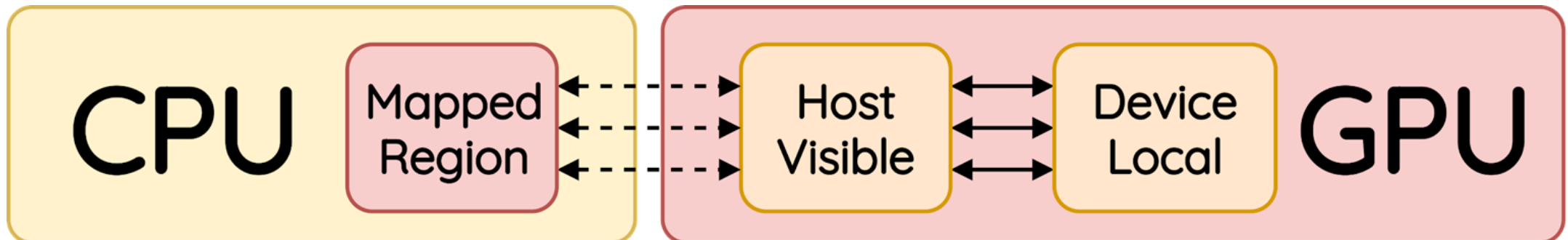
Memory Management

- Programmer responsible for allocations
- No dedicated index/vertex buffers
- Everything are just bytes in memory
- Single buffer can hold multiple resources
 - The right way!



Memory Types

- **Device Local**: invisible for CPU
 - Typically, there is all your data stored
- **Host visible | Host Coherent**: CPU visible
 - Great for data, which change a lot (e.g. uniform buffers)
- **Device Local | Host visible | Host Coherent**: ????



CommandBuffer



- Recording of rendering/compute/decode commands
- Linked to specific queue
- Primary/Secondary command buffers

```
// Create commandPools
auto poolCI = vk::CommandPoolCreateInfo()
    .setFlags(vk::CommandPoolCreateFlagBits::eResetCommandBuffer)
    .setQueueFamilyIndex(mQueueFamilyIndex);

mCommandPool = mDevice->createCommandPoolUnique(poolCI);

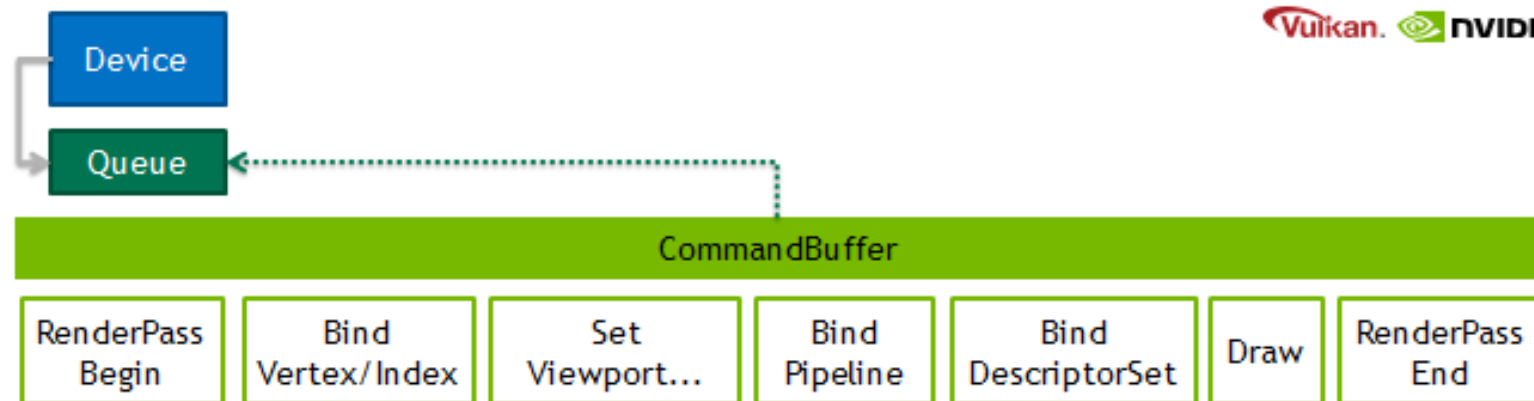
// Allocate command buffers
auto bufferCI = vk::CommandBufferAllocateInfo()
    .setLevel(vk::CommandBufferLevel::ePrimary)
    .setCommandPool(*commandPool)
    .setCommandBufferCount(kFramesInFlight);

mCommandBuffer = mDevice->allocateCommandBuffersUnique(bufferCI);
```

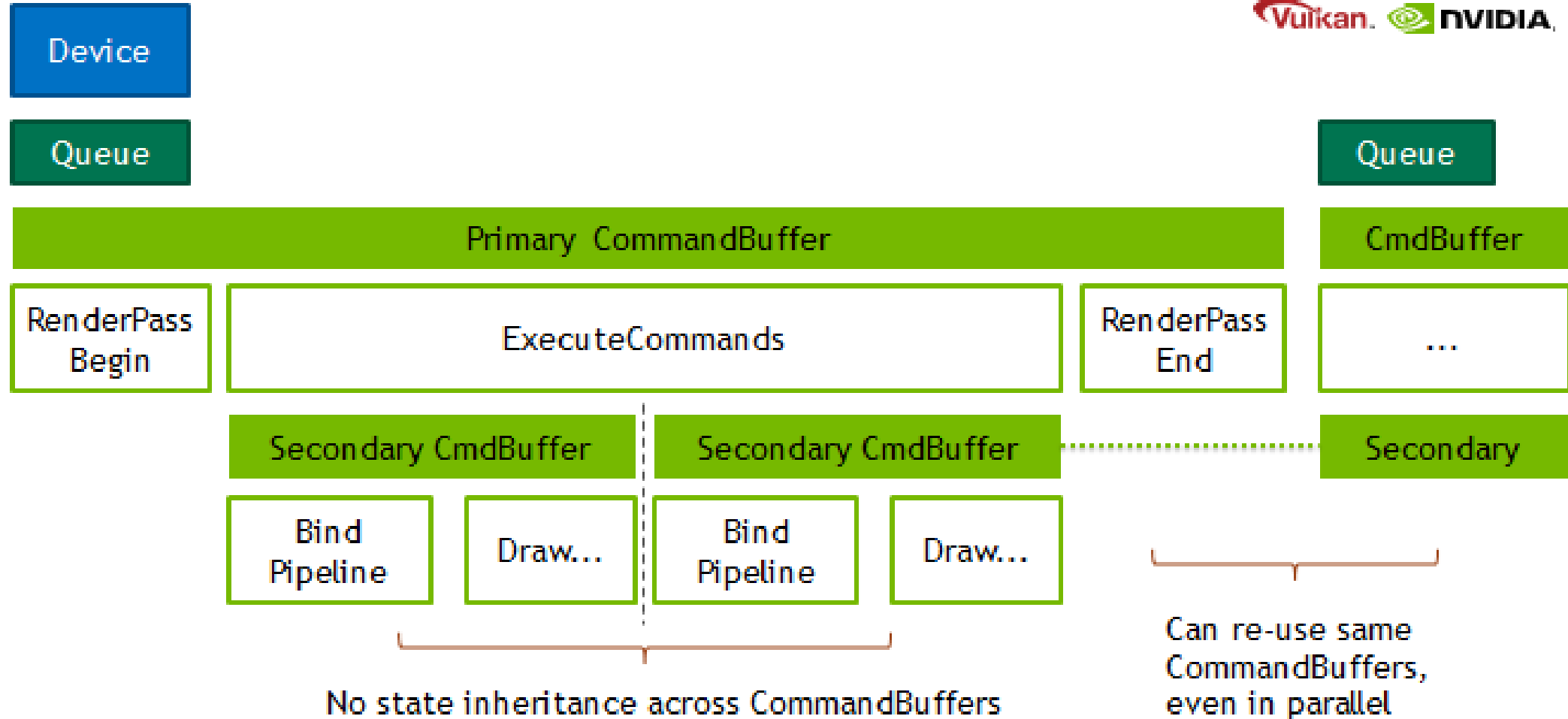
CommandBuffer Recording



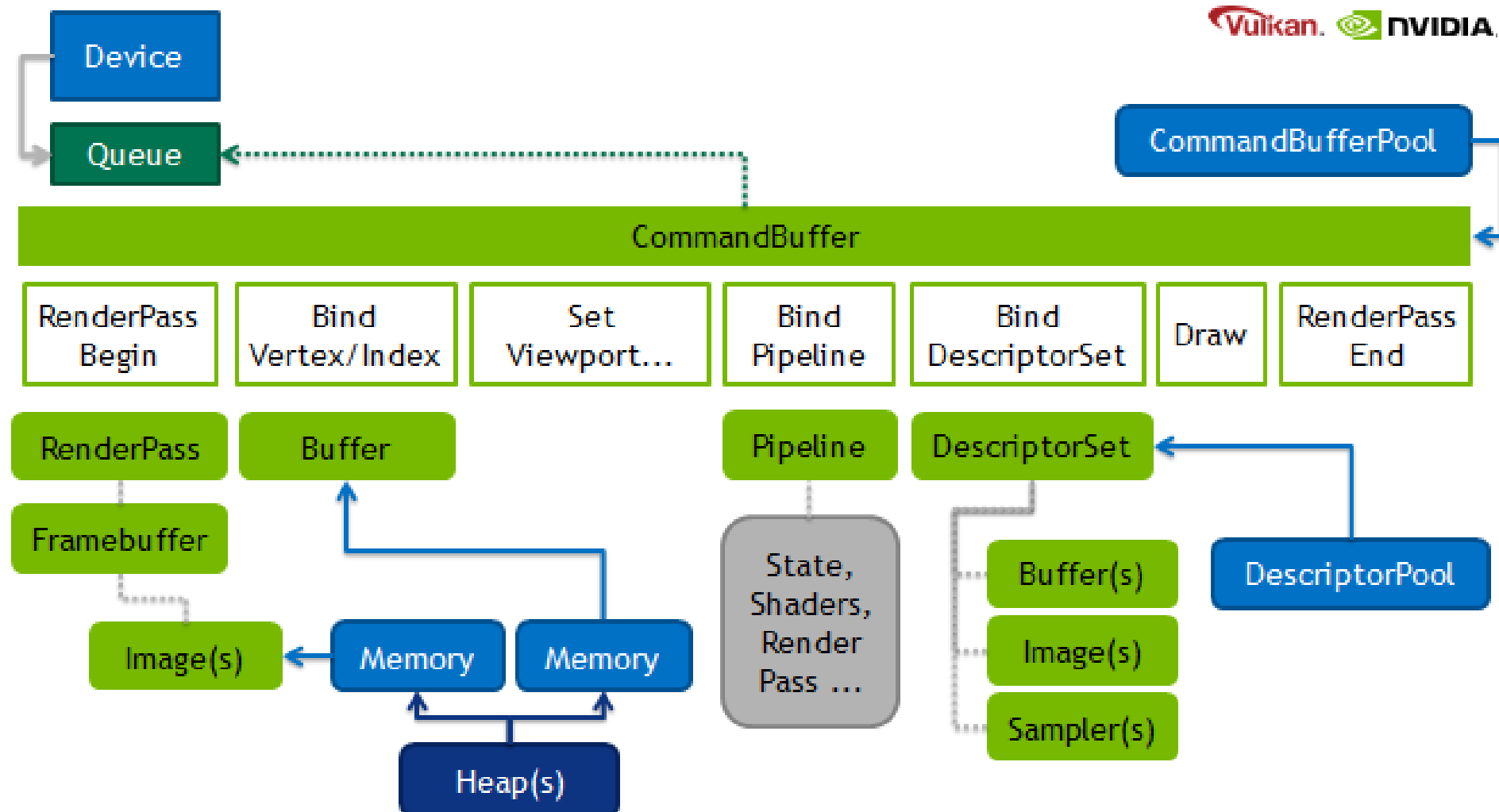
```
cmd.begin(vk::CommandBufferBeginInfo());  
cmd.beginRenderPass(renderpassCI, vk::SubpassContents::eInline);  
cmd.bindPipeline(vk::PipelineBindPoint::eGraphics, *mGraphicsPipeline);  
cmd.bindDescriptorSets(vk::PipelineBindPoint::eGraphics, *mPipelineLayout, 0, descriptorSets, nullptr);  
cmd.bindVertexBuffers(0, { vertexBuffer }, { 0 });  
cmd.bindIndexBuffer(indexBuffer, vertexCount * sizeof(Vertex), vk::IndexType::eUint32);  
cmd.draw(3, 1, 0, 0);  
cmd.endRenderPass();  
cmd.end();
```



Secondary CommandBuffer



Recap



Synchronization Basics



- Correct synchronization is very hard...
- How much?

Home

TobiasHector edited this page on 27 Jul 2017 · 3 revisions

Welcome to the Vulkan-Docs wiki!

This thing is fairly sparse at the moment, please feel free to contribute! Contents so far:

[Synchronization Examples](#)

Synchronization Basics

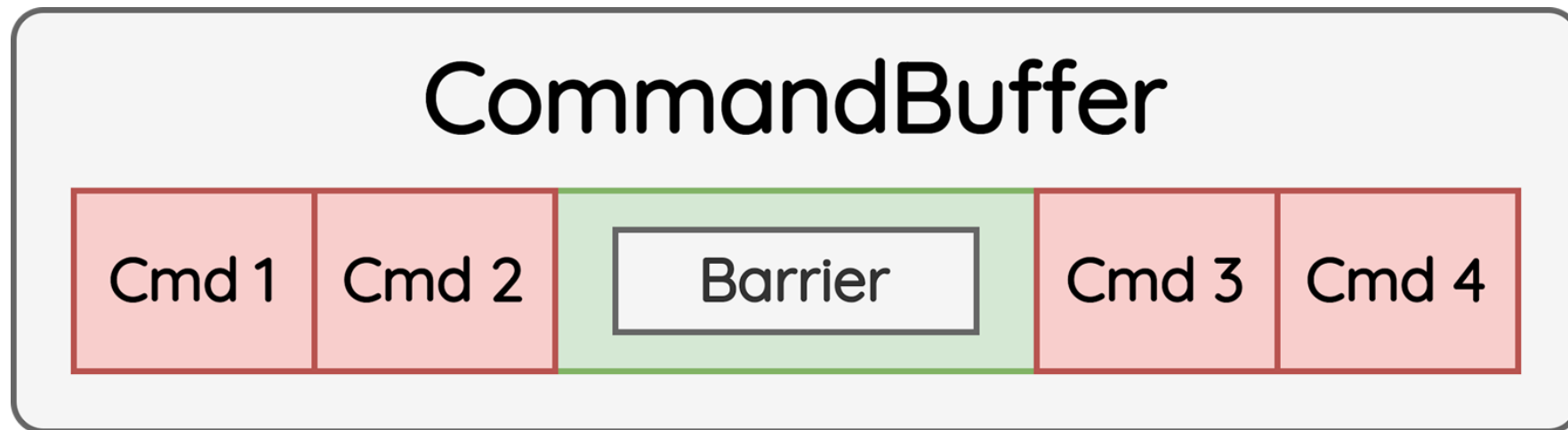


- Minimize GPU idling time, while preserving correct results
- **Under-synchronization:** memory corruption, wrong execution ordering, ...
- **Over-synchronization:** GPU stalling
- Synchronization primitives:
 - Semaphore
 - PipelineBarrier
 - Event
 - Fence
 - Subpass dependency

PipelineBarrier

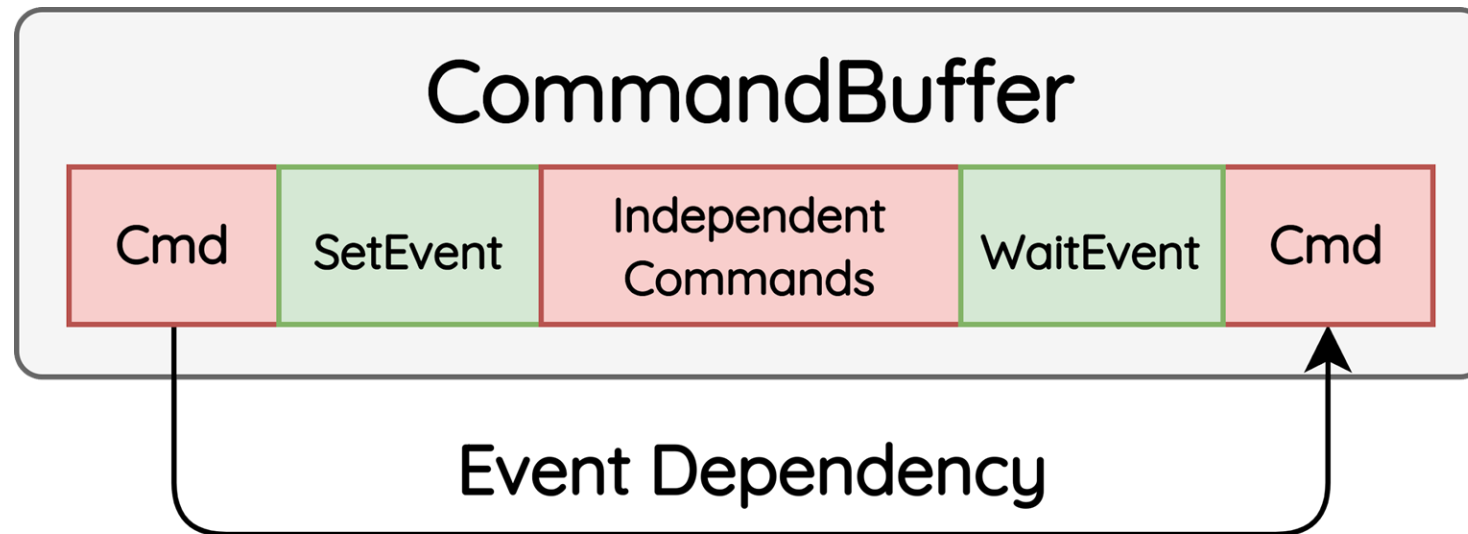


- Synchronization of commands within single queue
- Commands before barrier must finish, before commands after can start
- Typically, only execution/memory dependencies



Events

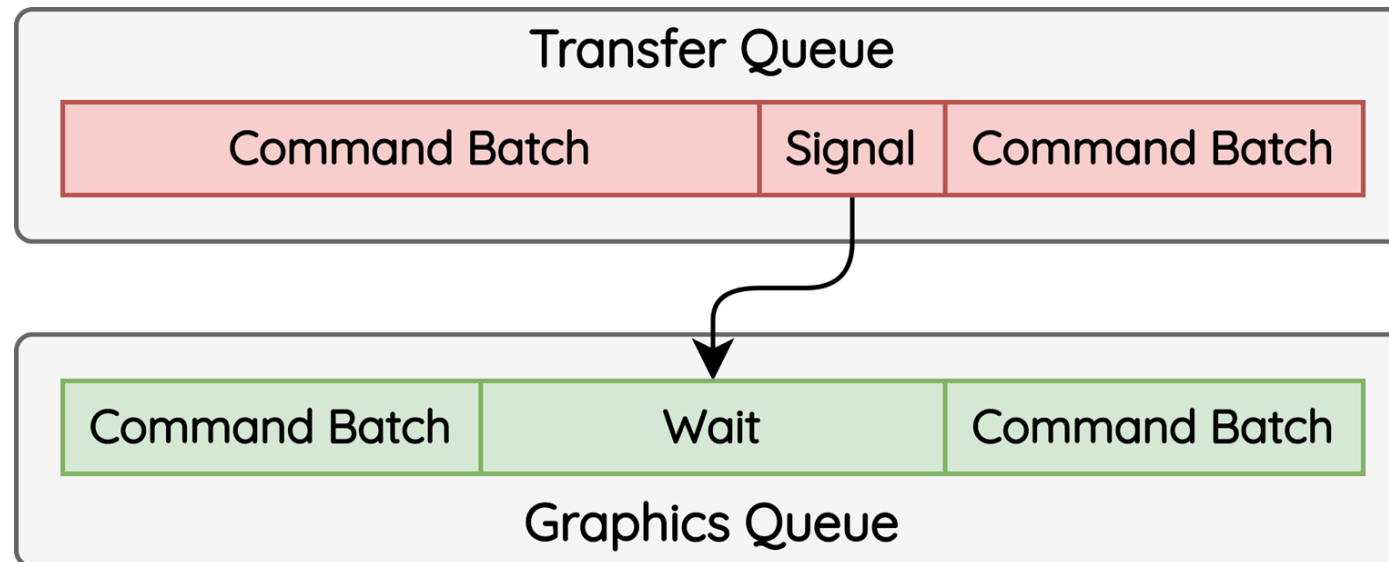
- Similar to pipeline barriers
- Used for executing independent work, which can hide latency



Semaphore



- Synchronization between queue operations
- Typically, synchronization between:
 - Graphics to present queue
 - Transfer to Graphics/Compute queue

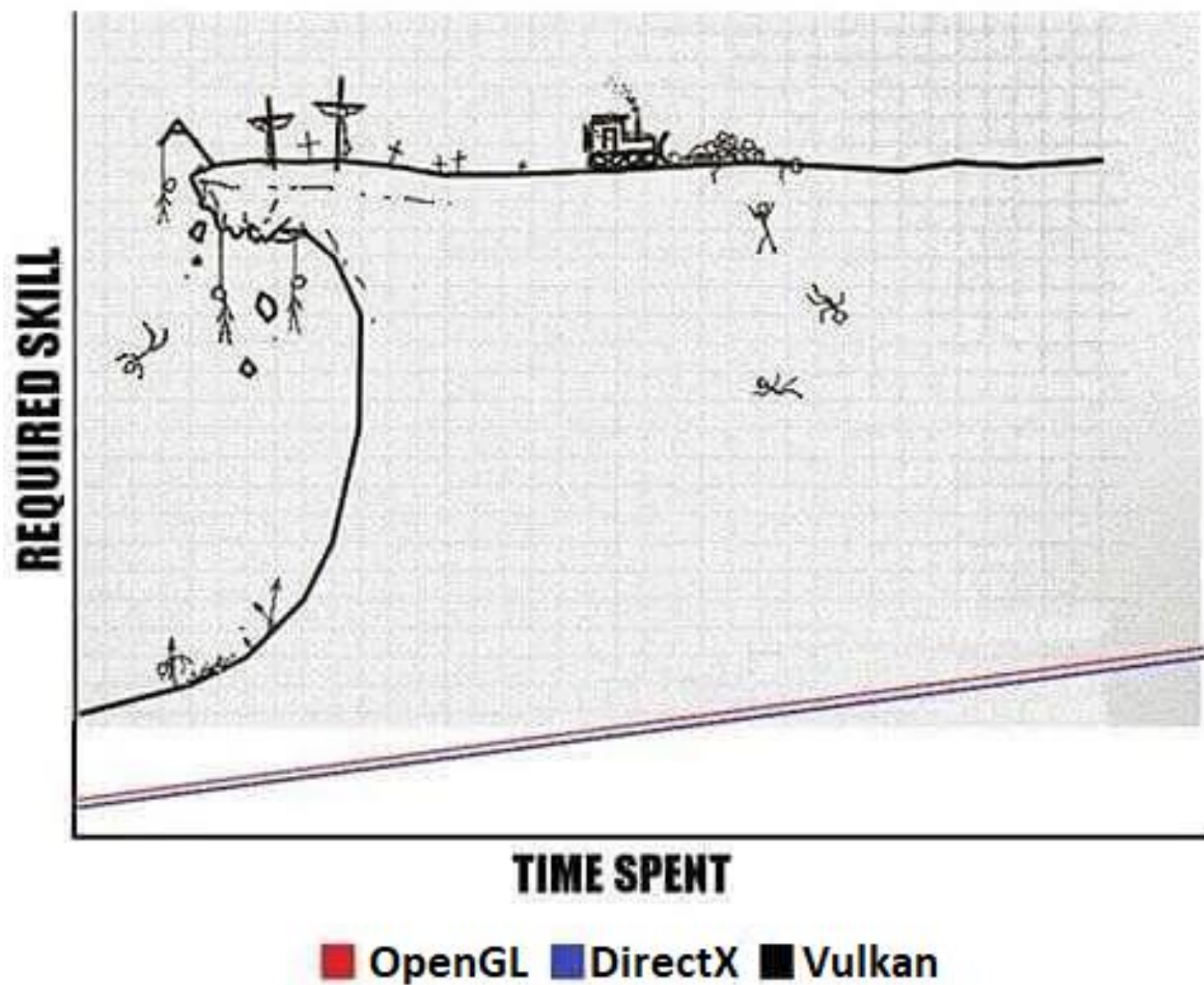


Fence



- Synchronization between CPU and GPU
- **Expensive**, synchronization between frames
 - Signalization of available resources (e.g. swapchain images in double/triple buffering)

GRAPHIC API LEARNING CURVE



Extensions & Libraries

VK_KHR_dynamic_rendering



- Promoted to Vulkan 1.3
- Allows single-pass Renderpass without creating Renderpass object
- Created 'on the fly' in command buffer recording

```
auto renderingCI = vk::RenderingInfo()  
    .setColorAttachments(attachments)  
    .setLayerCount(1)  
    .setPDepthAttachment(&depthAttachemnt)  
    .setRenderArea({ {0, 0}, mWindowExtent });
```

```
cmd.pipelineBarrier2(preDependencyI);  
cmd.beginRendering(renderingCI);  
cmd.bindPipeline(vk::PipelineBindPoint::eGraphics, *mDrawPipeline);  
cmd.setViewport(0, vk::Viewport(0, 0, mWindowExtent.x, mWindowExtent.y, 0, 1));  
cmd.draw(3, 1, 0, 0);  
cmd.endRendering();  
cmd.pipelineBarrier2(postDependencyI);
```

VK_KHR_push_descriptor



- Allows to write descriptors into command buffer

```
const auto layoutBindings = {  
    vk::DescriptorSetLayoutBinding()  
        .setBinding(0)  
        .setDescriptorCount(1)  
        .setDescriptorType(vk::DescriptorType::eCombinedImageSampler)  
        .setStageFlags(vk::ShaderStageFlagBits::eFragment),  
};  
  
const auto layoutCI = vk::DescriptorSetLayoutCreateInfo()  
    .setFlags(vk::DescriptorSetLayoutCreateFlagBits::ePushDescriptorKHR)  
    .setBindings(layoutBindings);  
  
mDescriptorSetLayout = mDevice->createDescriptorSetLayoutUnique(layoutCI);
```


VK_KHR_push_descriptor



- Allows to write descriptors into command buffer

```
auto writeSet = {  
    vk::WriteDescriptorSet()  
        .setDstSet(0)  
        .setDstBinding(0)  
        .setDescriptorCount(1)  
        .setDescriptorType(vk::DescriptorType::eCombinedImageSampler)  
        .setImageInfo({ ... } ),  
};  
  
cmd.pushDescriptorSetKHR(  
    vk::PipelineBindPoint::eGraphics,  
    pipelineLayout,  
    0, // First desc. set  
    writeSet  
);
```

VK_KHR_shader_object



- Removes usage of pipelines (allows permutation of shader stages on-fly)
- Able to bind shaders directly from command buffers
- Already in Nvidia beta drivers
- Cons:
 - Probably less performant
 - No raytracing shaders yet
 - Some pipeline states not present

```
typedef struct VkShaderCreateInfoEXT {  
    VkStructureType           sType;  
    const void*               pNext;  
    VkShaderCreateFlagsEXT    flags;  
    VkShaderStageFlagBits     stage;  
    VkShaderStageFlags        nextStage;  
    VkShaderCodeTypeEXT       codeType;  
    size_t                    codeSize;  
    const void*               pCode;  
    const char*               pName;  
    uint32_t                  setLayoutCount;  
    const VkDescriptorSetLayout* pSetLayouts;  
    uint32_t                  pushConstantRangeCount;  
    const VkPushConstantRange* pPushConstantRanges;  
    const VkSpecializationInfo* pSpecializationInfo;  
} VkShaderCreateInfoEXT;
```

VK_KHR_shader_object



- Removes usage of pipelines (allows permutation of shader stages on-fly)
- Able to bind shaders directly from command buffers
- Already in Nvidia beta drivers
- Cons:
 - Probably less performant
 - No raytracing shaders yet
 - Some pipeline states not present

```
// Fill struct
VkShaderCreateInfoEXT shaderInfo[2] = {
    ...
};

// Create Shader Objects
VkShaderEXT shaders[2];
vkCreateShadersEXT(device, 2, shaderInfo, NULL, shaders);

// When rendering, use created Shader Objects
VkShaderStageFlagBits stages[2] = {
    VK_SHADER_STAGE_VERTEX_BIT,
    VK_SHADER_STAGE_FRAGMENT_BIT
};
vkCmdBindShadersEXT(cmd, 2, stages, shaders);
```

Vulkan Memory Allocator (VMA)



- Removes boilerplate of Vulkan
- Manages allocations in “the good” way
- Need to create global *VmaAllocator*

```
VkBufferCreateInfo bufferInfo = {  
    VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO  
};
```

```
bufferInfo.size = 65536;  
bufferInfo.usage =  
    VK_BUFFER_USAGE_VERTEX_BUFFER_BIT |  
    VK_BUFFER_USAGE_TRANSFER_DST_BIT;
```

```
VmaAllocationCreateInfo allocInfo = {};  
allocInfo.usage = VMA_MEMORY_USAGE_AUTO;
```

```
VkBuffer buffer;  
VmaAllocation allocation;  
vmaCreateBuffer(  
    allocator,  
    &bufferInfo,  
    &allocInfo,  
    &buffer,  
    &allocation,  
    nullptr  
);
```

Other libraries with Vulkan support



- ImGui – GUI drawing
- SDL2 – Window & input handling
- Glslang – GLSL to SPIRV shader compiler
- Shaderc – Collection of tools and libraries for shader compilation
- Zink – OpenGL over Vulkan

Vulkan Video

Problems before Vulkan Video



- Options
 - System-specific
 - Vendor-specific
 - System AND vendor specific
- Some let you export frames to OpenGL/OpenCL
- No synchronization
- Weird formats

Platform API Availability [†]

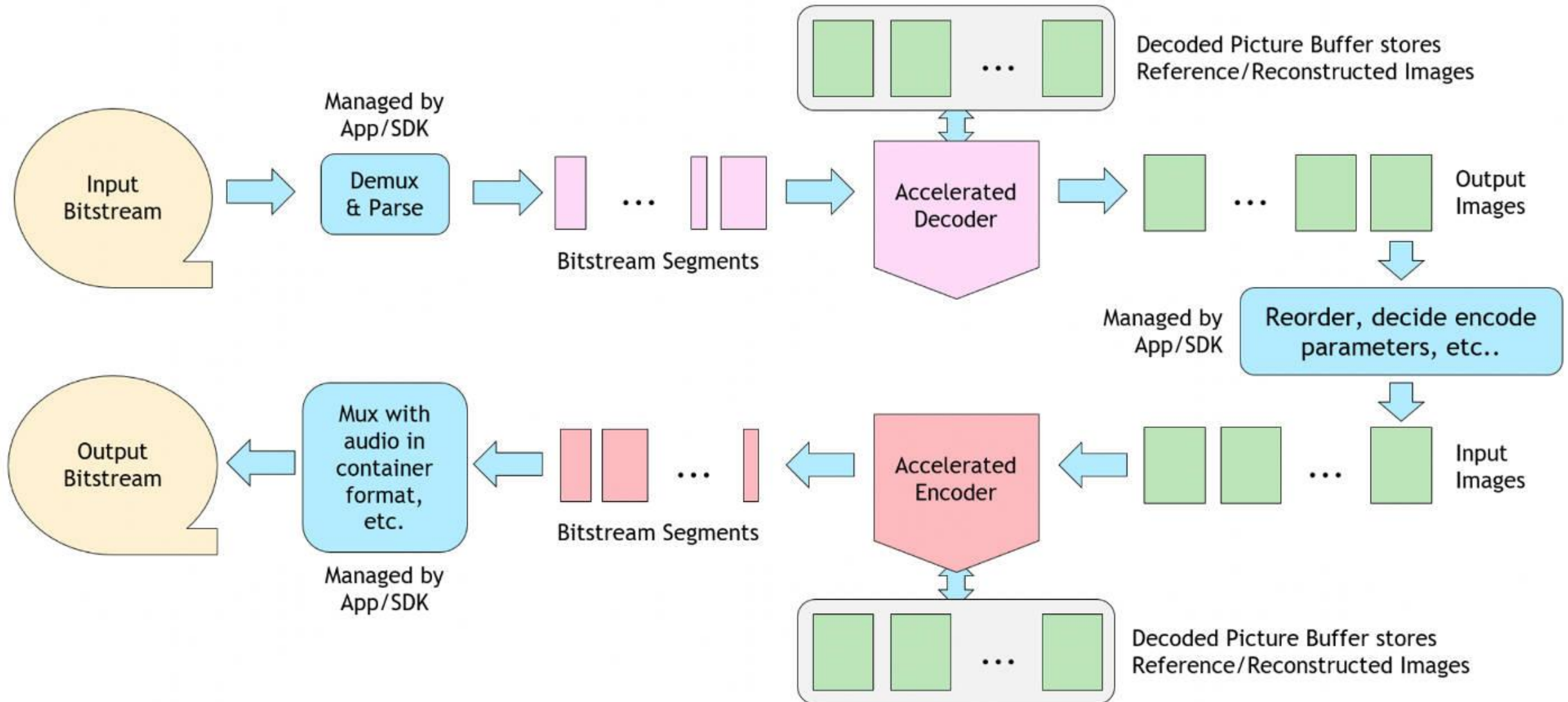
	Linux			Windows			Android	Apple		Other
	AMD	Intel	NVIDIA	AMD	Intel	NVIDIA		macOS	iOS	Raspberry Pi
AMF	N	N	N	Y	N	N	N	N	N	N
NVENC/NVDEC/CUVID	N	N	Y	N	N	Y	N	N	N	N
Direct3D 11	N	N	N	Y	Y	Y	N	N	N	N
Direct3D 9 (DXVA2)	N	N	N	Y	Y	Y	N	N	N	N
libmfx	N	Y	N	N	Y	N	N	N	N	N
MediaCodec	N	N	N	N	N	N	Y	N	N	N
Media Foundation	N	N	N	Y	Y	Y	N	N	N	N
MMAL	N	N	N	N	N	N	N	N	N	Y
OpenCL	Y	Y	Y	Y	Y	Y	P	Y	N	N
OpenMAX	P	N	N	N	N	N	P	N	N	Y
V4L2 M2M	N	N	N	N	N	N	P	N	N	N
VAAPI	P	Y	P	N	N	N	N	N	N	N
VDPAU	P	N	Y	N	N	N	N	N	N	N
VideoToolbox	N	N	N	N	N	N	N	Y	Y	N

Vulkan to the Rescue



- No compromises – low level enough
- Allows to use HW decode/encode queues
- Enables synchronization
- Able to work with original decoded images
- Encode is still provisional
- **Beta** drivers needed
- Extensions
 - VK_KHR_video_queue
 - VK_KHR_video_decode_queue
 - VK_KHR_video_decode_h264
 - VK_KHR_video_decode_h265

Transcoding



1st Attempt Implementation



- Vulkan reference manual
 - <https://registry.khronos.org/vulkan/>
- Only for those who worked on that extensions
 - Deep knowledge of Codecs/Hw/Vulkan
- **FAIL!**



2nd Attempt Implementation



- Official VK_Video samples
 - https://github.com/nvpro-samples/vk_video_samples
- Open-source samples
 - Except there is a small black-box
 - In the most important part
 - Parsing the packets
 - Filling Vulkan structs
- FAIL!



3rd Attempt Implementation



- Unofficial FFmpeg branch with Vulkan support
 - <https://github.com/cyanreg/FFmpeg>
- Actively developed
- In future will be merged into master
- Full of bugs
- Didn't work in the end
 - Developed/tested on linux
 - Nvidia was buggy
- **FAIL!**



Vulkan Video Reference



- A deep dive into Vulkan Video by Tony Zlatinski
 - https://www.youtube.com/watch?v=R5x6_nBRrv4
- Vulkan video with FFmpeg + RADV integration experiments
 - <https://themaister.net/blog/2023/01/05/vulkan-video-shenanigans-ffmpeg-radv-integration-experiments/>
- Official Vulkan Video Samples
 - https://github.com/nvpro-samples/vk_video_samples
 - <https://www.khronos.org/blog/khronos-finalizes-vulkan-video-extensions-for-accelerated-h.264-and-h.265-decode>

Useful resources



Big thanks to Ondřej Pavla for his slides

- Synchronization
 - <http://themaister.net/blog/2019/08/14/yet-another-blog-explaining-vulkan-synchronization/>
 - <https://github.com/KhronosGroup/Vulkan-Docs/wiki/Synchronization-Examples>
 - <https://www.youtube.com/watch?v=YkJ4hKCPjm0>
- Best-practices
 - <https://zeux.io/2020/02/27/writing-an-efficient-vulkan-renderer/>
 - <https://developer.nvidia.com/blog/vulkan-dos-donts/>
 - <https://developer.nvidia.com/vulkan-memory-management>
 - https://gpuopen.com/presentations/2019/Vulkanised2019_06_optimising_aaa_vulkan_title_on_desktop.pdf
- Samples
 - <https://saschawillems.de/>
 - <https://github.com/SaschaWillems>
- Tutorials
 - <https://vulkan-tutorial.com/>